Kingdom of Morocco
Sultan Moulay Slimane University
Superior School of Technology — Beni Mellal
Department of Computer Science

**Field: Cyber Security (CS)**

**Degree Program: Bachelor**

# Internship Report

Under the theme

# Implementation of a Security Operations Center (SOC) for Threat Detection and Response

**Prepared by:**

- *Intissar FARHOUN*
- *Mohammed KHALISS*

**Supervised by:**

*industrial Supervisor:* **Mlle.Chaimae lamachi**

*Academic Supervisor:* **Pr. Siham BAKKOURI**

*Internship period: from **01/04/2026** to **23/06/2026***

*Presented and defended on **[24/06/2026]**, before a Jury composed of:*

| | |
|---|---|
| Pr.SAID HAMANI | President |
| Pr. SIHAM BAKKOURI | Examiner |
| Pr. YASSIN MARAOUI | Examiner |

*Internship location:*

**GM-SOFT BENI Mellal**

*Academic year: 2025-2026*

# Acknowledgements

This internship has been a profoundly enriching phase of our academic and professional development, providing a bridge between theoretical learning and complex industrial application. We wish to express our sincere gratitude to all the individuals and institutions whose support made this endeavor successful. Foremost, we extend our deepest appreciation to the administrative and teaching staff of the Superior School of Technology of Beni Mellal (ESTBM) for their rigorous academic training, which provided the foundational knowledge necessary to undertake an engineering project of this scale.

Furthermore, we are immensely grateful to the management and technical teams at GM-Soft for welcoming us into their dynamic professional environment. The collaborative atmosphere fostered an ideal setting for immersion into the contemporary challenges of the cybersecurity sector. A special note of gratitude is owed to Mr. Saifeddine, our industrial supervisor, whose technical acumen, consistent patience, and constructive feedback were instrumental throughout the deployment of the SOC platform. His ability to clarify complex network architectures kept us motivated during the most demanding phases of the project. We also offer our heartfelt thanks to Pr. Siham BAKKOURI, our academic supervisor, whose rigorous oversight and valuable advice continually refined our analytical approach and ensured the academic integrity of our work. Finally, we thank our families and peers for their unwavering support and encouragement throughout our academic journey.

# List of Acronyms

| Acronym | Full Form |
|---|---|
| API | Application Programming Interface |
| ELK | Elasticsearch, Logstash, Kibana |
| ESN/SSII | IT Services Company (Digital Services / IT Engineering Company) |
| IDS | Intrusion Detection System |
| IoC | Indicator of Compromise |
| IPS | Intrusion Prevention System |
| MISP | Malware Information Sharing Platform |
| SIEM | Security Information and Event Management |
| SOAR | Security Orchestration, Automation and Response |
| SOC | Security Operations Center |

# List of Tables


1.1 GM-Soft — Key Corporate and Operational Facts . . . . . 11
1.2 Internship phases, primary activities, and corresponding deliverables . . . . . 13
1.3 Comprehensive list of technologies utilized in the project architecture . . . . . 15

2.1 Virtual machines provisioned within the Azure SOC laboratory . . . . . 18

3.1 Empirical performance metrics recorded across all attack simulations . . . . . 29
2 Environment variables for the SOC Report Generator . . . . . 63

# List of Figures


2.1 Global architectural topology of the SOC prototype at GM-Soft . . . . . 19

3.1 Performing a full vulnerability and port scan against the target system. . . . . 26
3.2 Simulating a high-velocity SSH brute-force attack against the target VM using invalid credentials. . . . . 27
3.3 Real-time detection of repeated SSH authentication failures originating from the attacker node. . . . . 27
3.4 File Integrity Monitoring alert triggered by the simulated malicious payload deployment. . . . . 28
3.5 Odoo brute force attack simulation showing HTTP 400 responses from the target. 28
3.6 Email notification with the automatically generated PDF report attached. . . . . 30
3.7 Cover page of the LLM-generated SOC report. . . . . 31
3.8 Real-time anomaly detection dashboard showing live anomaly grade (1.00) and confidence (1.08e-3). . . . . 32
3.9 Historical anomaly overview showing 5 anomaly occurrences with grades between 1.00 and 1.00. . . . . 33
3.10 Feature breakdown showing event$_{c}ountanomaliesovertimewithexpectedvalues.$ 33
3.11 Feature breakdown showing event$_{c}ountanomaliesovertimewithexpectedvalues.$ 33

# Abstract

This report details the end-to-end implementation of a Security Operations Center (SOC) prototype developed during an intensive internship at GM-Soft, an IT services company headquartered in Beni Mellal, Morocco. In response to the escalating sophistication and frequency of cyber threats targeting diverse enterprise environments, the core objective of this project was to design and deploy a centralized platform capable of ingesting, normalizing, analyzing, and responding to network and host-level security events in near real-time. To ensure broad accessibility and eliminate licensing overhead, the solution relies entirely on an open-source software stack. This ecosystem includes Wazuh functioning as the primary Security Information and Event Management (SIEM) engine, the ELK stack for high-performance log retention and visualization, Snort providing inline network Intrusion Prevention System (IPS) capabilities, pfSense serving as the perimeter firewall boundary, MISP acting as the threat intelligence clearinghouse, and Shuffle orchestrating automated incident response pipelines. The entire architecture was deployed within a Microsoft Azure virtualized cloud environment and rigorously validated through a series of simulated adversary scenarios, encompassing broad reconnaissance scanning, aggressive authentication brute-forcing, and sophisticated malware deployment. The empirical results definitively demonstrate that the integrated platform effectively detects anomalous behavior, issues prioritized alerts, and programmatically initiates automated containment responses within a five-second operational window. This implementation drastically minimizes the organization's risk exposure and significantly enhances its proactive security posture while remaining highly cost-effective.

**Keywords:** SOC, SIEM, Wazuh, cybersecurity, automation.

# Contents

**Acknowledgements** **1**

**List of Acronyms** **2**

**List of Tables** **3**

**List of Figures** **4**

**Abstract** **5**

**Table of Contents** **7**

**General Introduction** **8**

**Chapter 1 :Organizational Context and Project Definition** **10**

Introduction . . . . . 10

1 Presentation of the Host Company . . . . . 10

2 Project Context, Objectives, and Methodology . . . . . 12

3 Technologies and Tools Deployed . . . . . 14

Conclusion . . . . . 16

**Chapter 2 :Theoretical Framework and Practical Implementation of the SOC** **17**

Introduction . . . . . 17

1 Centralized Event Management: SIEM Architecture . . . . . 19

2 Perimeter Defense: Active Intrusion Prevention and Passive Network Monitoring 21

3 Intelligence Integration, Incident Response, and SOAR Orchestration . . . . . 22

Conclusion . . . . . 24

**Chapter 3 :Validation, Testing, and Operational Results** **25**

Introduction . . . . . 25

1 Simulated Attack Scenarios . . . . . 25

1.1 Network Reconnaissance (Nmap Scan) . . . . . 25

1.2 SSH Brute-Force Attack . . . . . 26

1.3 Malware Deployment via Path Traversal . . . . . 27

1.4 Odoo ERP Attack Simulation . . . . . 28

2 Performance Evaluation and Operational Insights . . . . . 29
3 LLM-Generated Report Results . . . . . 30
4 OpenSearch Anomaly Detection Results . . . . . 32
Conclusion . . . . . 34

**General Conclusion** **35**
Perspectives . . . . . 36

**Bibliography** **37**

**List of Appendices** **38**

**Appendix 1: Wazuh + ELK: Detailed Installation Steps** **39**

**Appendix 2: OpenSearch Anomaly Detection - Detector Configuration Script** **40**

**Appendix 3: LLM-Powered SOC Report Generator** **46**

# General Introduction

As the global digital landscape rapidly evolves, the frequency, scale, and sophisticated nature of cyberattacks present an unprecedented challenge to organizational infrastructure. Traditional, perimeter-centric defense mechanisms—such as legacy packet-filtering firewalls and signature-based antivirus software—are no longer capable of providing sufficient protection against modern advanced persistent threats. Malicious actors increasingly employ highly evasive techniques designed to bypass static defenses, dwell silently within networks, and execute devastating data exfiltration or ransomware campaigns. In response to this shifting threat paradigm, the implementation of a Security Operations Center (SOC) has transitioned from being an enterprise luxury to an absolute strategic necessity. A SOC acts as the centralized nervous system of an organization's defensive posture, continuously aggregating telemetry from heterogeneous sources, correlating diverse alerts to identify complex attack patterns, and facilitating a coordinated, immediate incident response.

GM-Soft, a versatile IT services company operating across both Moroccan and German markets, encounters these profound security challenges directly. As a provider of custom software development, enterprise resource planning systems, and IT consulting, the company manages a highly diverse technological ecosystem. Securing both its internal proprietary infrastructure and the external services it provides to its clientele requires robust, centralized visibility. This internship project was consequently initiated with a clear mandate: to design, engineer, and deploy a fully functional SOC prototype tailored to GM-Soft's operational realities. Recognizing the budget constraints often faced by small and medium-sized enterprises, the architecture was explicitly designed to leverage exclusively open-source tools, ensuring the final solution remains exceptionally cost-effective while delivering enterprise-grade security capabilities.

The resulting integrated platform brings together several critical pillars of modern cybersecurity. It utilizes a centralized SIEM for comprehensive log management and behavioral analysis, an active network intrusion prevention system for deep traffic inspection, a dedicated threat intelligence platform for the ingestion of global compromise indicators, and an advanced orchestration engine to automate repetitive incident response workflows. By automating the containment of identified threats, the platform seeks to minimize the Mean Time to Respond (MTTR), vastly reducing the window of opportunity available to potential attackers.

Adhering strictly to the Anglo-Saxon methodological structure, this comprehensive report is articulated around three primary chapters. **Chapter 1** establishes the foundational context of the project. It provides an in-depth presentation of the host company, delineates the precise

operational challenges that necessitated the SOC initiative, and outlines the strategic objectives and Agile-inspired methodology that governed the project's execution. **Chapter 2** serves as the technical nucleus of the report, uniquely synthesizing both the theoretical frameworks and the practical, hands-on implementations of each distinct SOC component within the cloud architecture. Finally, **Chapter 3** is dedicated to the empirical validation of the deployed platform. It details the execution of multiple simulated cyberattack scenarios, presents the system's quantitative performance metrics, and critically evaluates the operational successes and ongoing limitations of the newly established defensive infrastructure.

# Chapter 1

# Organizational Context and Project Definition

## Introduction

This initial chapter is dedicated to framing the comprehensive organizational and operational environment within which this internship took place. Before exploring complex cybersecurity architectures, it is essential to understand the specific business ecosystem that these architectures are designed to protect. The chapter begins by providing an extensive presentation of the host company, GM-Soft, highlighting its operational scope, international strategic partnerships, and diverse software development expertise. Subsequently, the narrative transitions into identifying the core technological vulnerabilities that prompted this initiative. It formally establishes the project's strategic objectives and meticulously details the phased, Agile-inspired methodology that structured the project's execution from initial discovery to final validation.

## 1. Presentation of the Host Company

To fully appreciate the critical nature of this cybersecurity initiative, one must first examine the organizational structure and business imperatives of the host company. GM-Soft operates prominently as an IT Services Company, maintaining a highly active presence across both the Moroccan and German technology markets. The enterprise specializes in the bespoke design, development, and implementation of customized IT solutions that are carefully tailored to address highly specific client requirements. By consistently leveraging modern programming frameworks and innovative consulting approaches, GM-Soft actively supports a wide array of businesses in their digital transformation journeys, ultimately enhancing their overall operational efficiency and market competitiveness.

A defining characteristic of GM-Soft's business model is its robust international orientation, anchored by a strategic partnership with the German firm WBG-Software GmbH. This cross-border collaboration is instrumental in aligning GM-Soft's internal development methodologies with rigorous European engineering standards. Furthermore, this partnership facilitates seamless entry into the European market, allowing the company to expand its client base while main-

taining a highly capable and cost-effective operational headquarters in Beni Mellal, Morocco. This dual-market presence necessitates a highly resilient IT infrastructure capable of supporting international data flows securely.

**Table 1.1:** GM-Soft — Key Corporate and Operational Facts

| Attribute | Value |
|---|---|
| Company type | IT Services Company |
| Operating Regions | Morocco and Germany |
| Headquarters | Beni Mellal, Morocco |
| Specialization | Custom software, cybersecurity, IT consulting |
| German partner | WBG-Software GmbH (http://www.wbg-software.de) |
| Partner contact | ralf.hoeser@wbg-software.de |

*Source: GM-Soft internal corporate documentation (2026).*

The scope of services offered by GM-Soft is exceptionally comprehensive, covering the entire lifecycle of software engineering. The company fields dedicated teams for desktop application development, utilizing Microsoft-centric frameworks such as Windows Forms, WPF, and WCF. Concurrently, their web development division architectures complex platforms using ASP.NET MVC, Java EE, and modern PHP environments, alongside popular Content Management Systems like Drupal and Joomla. The mobile development sector further expands this technological footprint by engineering cross-platform applications using HTML5, Android native tools, and Sencha Touch. To support these applications, the database management teams operate a multitude of systems including Microsoft SQL Server, Oracle Database, and MySQL.

To serve specific industry verticals, GM-Soft has also engineered a suite of proprietary software solutions. These include GMSEC, a comprehensive civil status management system utilized by local municipalities; dedicated Enterprise Resource Planning (ERP) systems optimized for both general small-to-medium enterprises and highly regulated pharmaceutical companies; automated resume data extraction tools leveraging intelligent algorithms; and systems designed for the automated processing of electronic invoices.

This vast, incredibly diverse technological expertise and expansive software portfolio intrinsically create a massive digital attack surface. Each new framework, database technology, and client application introduces potential vulnerabilities into the ecosystem. Consequently, securing this varied array of internal infrastructure and client-facing technologies served as the primary motivation for developing robust, internal cybersecurity capabilities through the establishment of a dedicated Security Operations Center.

# 2. Project Context, Objectives, and Methodology

Building directly upon the understanding of GM-Soft's expansive technological landscape, this section clearly defines the operational gaps that left the organization vulnerable, outlines the precise, measurable objectives established to rectify these vulnerabilities, and explains the structured methodology employed to execute the project successfully.

Prior to the commencement of this internship, a preliminary audit of GM-Soft's infrastructure revealed several distinct operational security challenges. Foremost among these was the highly decentralized nature of their logging mechanisms. System events, application errors, and network traffic data were generated and stored in isolated silos across various servers and firewall interfaces. This fragmentation entirely prevented unified network visibility and cross-platform event correlation. As a direct consequence, the organization suffered from delayed incident detection capabilities; sophisticated, low-noise threat actors could potentially navigate the network undetected simply because no single system was configured to correlate disparate anomalies. Furthermore, the absence of an automated incident response framework meant that any active threat containment required manual intervention by network administrators. This reliance on human reaction speed significantly increased the window of risk exposure, allowing potential attacks to propagate before they could be manually neutralized.

To comprehensively address these systemic vulnerabilities, the primary objective of this internship was established: the design, engineering, and deployment of a complete SOC architecture tailored specifically to GM-Soft's environment. A crucial constraint and secondary objective was the strict utilization of open-source components. This ensured the resulting platform would incur zero software licensing fees, making it an economically viable model that GM-Soft could subsequently package and offer to their own SME clients. The project demanded the deployment of these tools within a virtualized Microsoft Azure cloud environment, requiring the careful configuration of cloud networking, virtual subnets, and robust interoperability between diverse Linux and BSD-based systems.

Beyond simple deployment, the project held advanced operational objectives. The platform needed to actively ingest external threat intelligence feeds from global cybersecurity communities, ensuring the local detection rules were continuously updated against emerging threats. Most ambitiously, the project aimed to develop fully automated response workflows utilizing a Security Orchestration, Automation, and Response (SOAR) platform. The defined performance target for these workflows was achieving a Mean Time to Respond (MTTR) of under five seconds, allowing the system to dynamically block attacking IP addresses without requiring any prior human authorization.

**Table 1.2:** Internship phases, primary activities, and corresponding deliverables

| Phase | Duration | Primary Activity | Key Deliverable |
|---|---|---|---|
| Discovery | Weeks 1–2 | Architecture audit | System blueprint |
| Deployment | Weeks 3–5 | Platform installation | Configured cloud VMs |
| Integration | Weeks 6–8 | System interconnection | Operational SOC pipeline |
| Testing & Docs | Weeks 9–10 | Adversary simulation | Final test report |

*Source: Personal elaboration based on the project management timeline.*

To navigate the complexity of deploying multiple interrelated security systems, an iterative, Agile-inspired methodology was adopted. The project was meticulously structured across four distinct phases spanning a ten-week period. The initial discovery phase was dedicated to mapping the existing infrastructure, understanding traffic flows, and drafting a comprehensive architecture blueprint. The subsequent deployment phase focused heavily on systems engineering, involving the sequential provisioning, baseline configuration, and hardening of the required virtual machines within the Azure cloud.

The third phase represented the most technically challenging period, focusing entirely on systems integration. This involved establishing secure, encrypted log forwarding tunnels, configuring complex Application Programming Interface (API) integrations between the threat intelligence platform and the SIEM, and establishing the webhook communication channels required for the SOAR orchestrator.To further augment the SOC's analytical capabilities, a Large Language Model (LLM) was integrated into the automation pipeline. This model, accessed via the Groq API, processes aggregated alerts from the Wazuh SIEM and generates structured, natural-language security reports that include attack classification, business impact assessment, and step-by-step remediation plans. In parallel, the native Anomaly Detection plugin of OpenSearch was operationalized to provide unsupervised machine-learning-based detection of deviations from established behavioral baselines. Custom detectors were configured on key metrics, such as failed authentication attempts, web error rates, and unusual access patterns, enabling the system to surface subtle, previously unseen attack patterns and zero-day attempts that rule-based correlation alone might miss. The LLM-generated reports now incorporate these anomaly scores alongside signature-based alerts, delivering a holistic view that blends deterministic rules with probabilistic risk indicators. This dual enhancement significantly strengthened the SOC's proactive detection posture and enriched the automated decision-making context for the SOAR orchestration layer.In addition to the operationalized LLM and anomaly detection capabilities, the SOC architecture was designed to incorporate a dedicated incident-response and forensics layer built around TheHive, Cortex, and a supplemental ELK (Elasticsearch, Logstash, Kibana) stack. TheHive was intended to serve as the central incident-management platform, enabling collaborative case tracking, while Cortex would provide automated observables anal-

ysis through its extensive library of analyzers and responders. The auxiliary ELK deployment was planned to ingest and correlate non-Wazuh log sources, offering extended visibility and long-term retention for forensic investigations. However, during the integration phase, several unresolved bugs (particularly related to authentication handshakes between Cortex and TheHive, index mapping conflicts in the custom ELK data streams, and erratic webhook delivery from the SOAR layer) prevented these components from reaching production stability. As of this writing, the incident-response layer remains in an active debugging and re-configuration state, with the resolution of these issues prioritized for the next maintenance sprint. Once stabilised, this trio will complete the SOC's defense-in-depth by providing structured case management, automated artifact enrichment, and deep historical log analysis, fully complementing the real-time detection and LLM-driven reporting already in place. The final two weeks were dedicated to rigorous quality assurance. This testing phase involved executing live adversary simulations against the platform, carefully tuning the detection rules to suppress false positives, measuring the automated response times, and finalizing the comprehensive technical documentation required for the long-term maintenance of the SOC.

# 3. Technologies and Tools Deployed

The successful implementation of the SOC prototype relied on a carefully selected ecosystem of open-source technologies, each chosen for its specific capabilities and seamless integration with the overall architecture. This section provides a comprehensive overview of all tools deployed within the Azure cloud environment, categorized by their primary function and role in the security pipeline.

**Table 1.3:** Comprehensive list of technologies utilized in the project architecture

| Tool | Category | Purpose in the Project |
|---|---|---|
| Microsoft Azure | Cloud platform | Infrastructure hosting and virtual networking |
| Wazuh 4.7 | SIEM / XDR | Log collection, correlation, and alerting |
| pfSense 2.7 | Firewall | Perimeter filtering and IP blocking |
| Snort 3 | IDS / IPS | Signature-based network traffic analysis (inline blocking) |
| Zeek | NSM / IDS | Passive network traffic analysis and protocol logging (HTTP, DNS, SSL, etc.) |
| MISP | Threat Intel | IoC aggregation and rule enrichment |
| Shuffle | SOAR | Automated incident response orchestration |
| VirusTotal API | Enrichment | External IP reputation lookup |
| Hydra & Nmap | Offensive Testing | Simulation of brute-force and port scanning |
| Docker Compose | Containerization | Deployment mechanism for Shuffle |
| Python 3 | Scripting | Custom MISP-to-Wazuh synchronization scripting |
| Groq API (LLaMA 3.3-70B) | AI/LLM | Automated alert classification and remediation plan generation |
| OpenSearch Anomaly Detection | ML/Analytics | Unsupervised detection of brute force, path traversal, and web scanning |
| TheHive | Incident Response | Centralized case management and incident tracking |
| Cortex | Threat Intel | Automated observables enrichment and analysis |
| Secondary ELK Stack | Log Management | Forensic investigation and non-Wazuh log correlation |

*Source: Personal elaboration detailing the deployed technology stack.*

This structured, phased approach ensured that each technological component was fully func-

tional, stable, and locally verified before it was integrated into the broader, highly interdependent architectural pipeline. This methodological discipline was crucial in mitigating the risk of cascading systemic failures during the final integration and testing phases.

## Conclusion

In summary, this foundational chapter has provided the essential context necessary to understand the scope and intent of the internship. By detailing GM-Soft's expansive, dual-market technological landscape, the inherent necessity for centralized security monitoring was clearly established. The identified operational vulnerabilities directly informed the project's ambitious objectives: constructing a fully automated, open-source SOC capable of machine-speed incident response. With the precise goals defined and the phased methodology outlined, the operational stage is fully set. The subsequent chapter will delve into the technical core of the report, exploring both the underlying theoretical cybersecurity mechanisms and the complex practical implementation of the chosen SOC components within the Azure cloud environment.

# Chapter 2

# Theoretical Framework and Practical Implementation of the SOC

## Introduction

Having established the organizational imperatives in the previous chapter, this central section details the core engineering efforts of the internship. Strictly adhering to the Anglo-Saxon academic structure, this chapter seamlessly integrates theoretical exploration with practical application. Rather than isolating abstract cybersecurity theory from the reality of systems administration, each subsequent section focuses on a specific defensive domain—centralized monitoring, active perimeter defense, and automated orchestration. Within each domain, the chapter first elucidates the underlying theoretical mechanisms before immediately detailing exactly how those theories were practically implemented, configured, and integrated within GM-Soft's Microsoft Azure cloud infrastructure.

Before exploring the individual components, it is critical to understand the overarching architectural design that governed their deployment. The entire SOC platform was instantiated within the Microsoft Azure cloud, a decision that mirrored realistic enterprise network topologies while accommodating the necessity for rapid scalability and resource isolation. The architecture was meticulously designed around the principle of defense-in-depth, recognizing that no single security measure is infallible.

**Table 2.1:** Virtual machines provisioned within the Azure SOC laboratory

| VM Hostname | Operating System | Primary Architectural Role |
|---|---|---|
| Wazuh-Manager | Ubuntu 20.04 | Central SIEM: log ingestion, correlation, dashboards |
| MISP-Intel | Ubuntu 20.04 | Threat intelligence aggregation platform |
| Shuffle-Orchestrator | Ubuntu 20.04 | SOAR orchestrator + containerized services (Shuffle, TheHive, Cortex, ELK, Zeek) |
| Odoo-ERP-Target | Ubuntu 22.04 | Odoo ERP application server (target for web attacks) |
| Ubuntu-Target | Ubuntu 22.04 | Target endpoint equipped with Wazuh agent |
| Kali-Attacker | Kali Linux 2024.1 | Isolated adversary node for attack simulation |

*Source: Personal elaboration based on Azure deployment specifications.*

This defense-in-depth approach layered multiple protective mechanisms. The perimeter layer, governed by pfSense and Snort, acted as the primary network boundary, filtering hostile traffic before it could interact with internal services. Complementing this active inline protection, a dedicated Zeek sensor was deployed to perform passive, deep packet inspection, generating rich protocol logs (HTTP, DNS, SSL, etc.) for enhanced network visibility without interfering with traffic flows. Conversely, the endpoint layer, utilizing Wazuh security agents, provided deep, granular visibility into host-level activities such as file modifications and privilege escalations. All telemetry generated by these distinct layers—including pfSense firewall logs, Snort IPS alerts, Zeek protocol metadata, and Wazuh agent telemetry—converged centrally at the SIEM, where it was enriched by external threat intelligence from MISP and acted upon by the orchestration engine. The SOAR layer, running on the Shuffle-Orchestrator VM within Docker containers, automated incident response workflows while TheHive and Cortex provided structured case management and observables enrichment. The secondary ELK Stack ensured long-term forensic log retention and advanced data visualization, completing the comprehensive security pipeline.

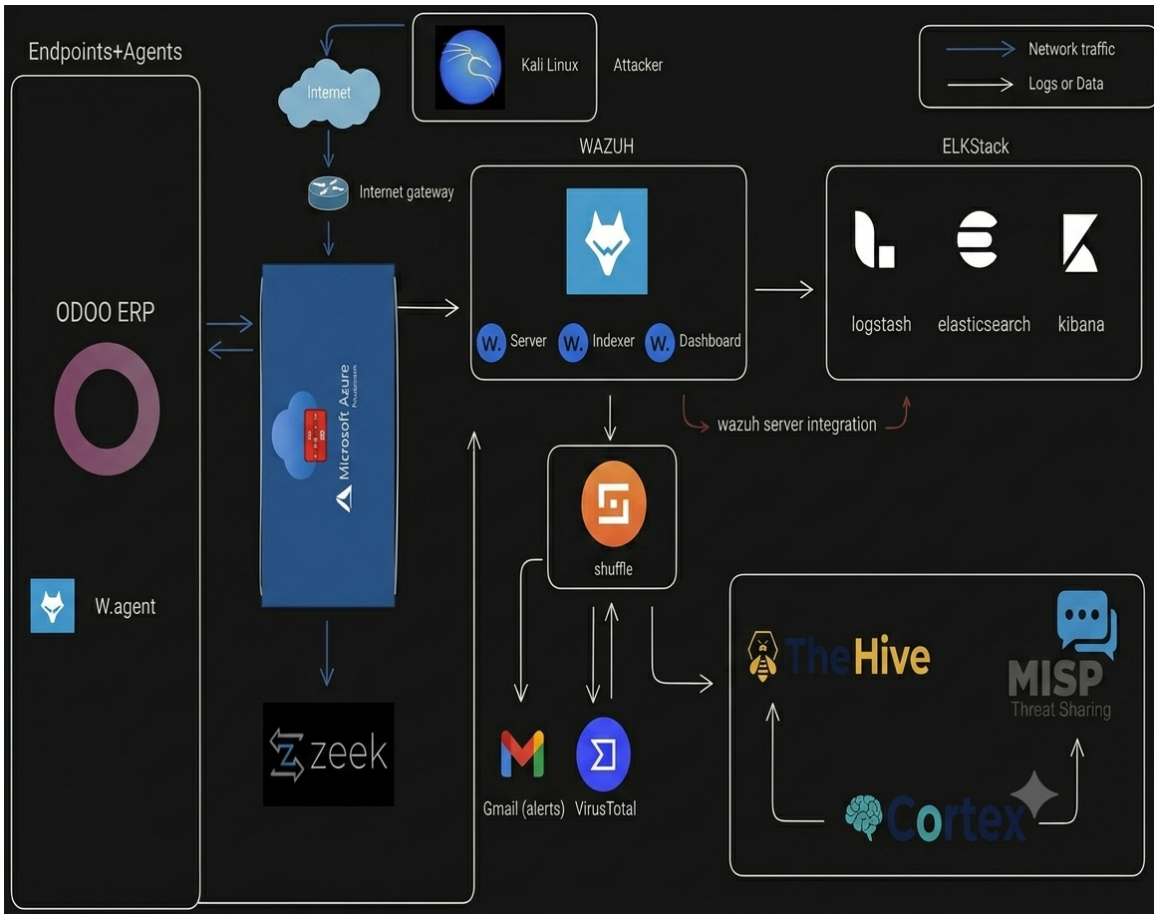



**Figure 2.1:** Global architectural topology of the SOC prototype at GM-Soft

*Source: Personal elaboration representing the deployed cloud environment.*

The diagram above illustrates these complex data flows. Solid vectors represent raw network traffic traversing the perimeter, whereas dashed vectors represent the secure, encrypted transmission of formatted log telemetry and API communications between the internal SOC components. The architecture comprises dedicated VMs for Wazuh (central SIEM), MISP (threat intelligence), and pfSense (perimeter firewall), while a single orchestration VM hosts containerized services including Shuffle (SOAR), TheHive (incident management), Cortex (observables enrichment), a secondary ELK Stack (forensic log retention), and Zeek (passive network protocol analysis). The target environment includes an Odoo ERP server and additional Ubuntu/Windows endpoints equipped with Wazuh agents, with a Kali Linux node serving as the isolated attacker for simulation purposes. With this global blueprint established, the subsequent sections will dissect the theoretical and practical implementation of each constituent layer.

# 1. Centralized Event Management: SIEM Architecture

A fundamental pillar of any modern Security Operations Center is its ability to aggregate, normalize, and analyze immense volumes of disparate data generated across an enterprise. This

section explores the theoretical premise of Security Information and Event Management (SIEM) systems before detailing the practical deployment of the chosen open-source solution.

Theoretically, a SIEM platform represents the convergence of two distinct disciplines: Security Information Management (SIM) and Security Event Management (SEM). The SIM component focuses on the long-term, secure archival and indexing of log data, which is essential for forensic investigations and maintaining regulatory compliance. Conversely, the SEM component is highly dynamic, providing real-time log correlation, complex rule-matching, and visual dashboarding. Modern SIEM architectures operate by deploying lightweight software agents across all network endpoints. These agents act as localized sensors, securely collecting telemetry—such as system performance metrics, authentication success and failure logs, and file integrity hashes—and transmitting this data back to a central management server. The central manager then normalizes this chaotic raw data against predefined decoders, translating it into a uniform format. This normalized data is subsequently evaluated through an extensive ruleset designed to identify complex behavioral patterns indicative of malicious activity, triggering alerts when defined thresholds are breached.

In practical application, the open-source Wazuh platform, functioning in tandem with the ELK stack (Elasticsearch, Logstash, Kibana), was selected to serve as the SIEM engine for the GM-Soft prototype. The entire system was deployed on an Ubuntu 20.04 virtual machine hosted within the Azure environment. The deployment process was initiated by utilizing Wazuh's automated installation scripts to systematically provision the indexer cluster, the central server component, and the analytical dashboard interface. Following the stabilization of the core installation, security agents were deployed across the laboratory's designated Windows Server and Linux target endpoints. To ensure communication security, these agents were enrolled using unique cryptographic keys generated via the manager's deployment API, guaranteeing that all transmitted telemetry was mutually authenticated and encrypted via TLS.

To tailor the platform to the specific operational realities of the environment, significant practical engineering was undertaken beyond the baseline installation. The core configuration files were heavily modified to establish listeners capable of ingesting external syslog traffic utilizing UDP port 514. Furthermore, custom correlation rules formulated in XML were engineered to detect precise anomalies highly relevant to modern attack vectors. For instance, specific rules were crafted to identify repeated Secure Shell (SSH) authentication failures exceeding five attempts within a sixty-second window, to detect anomalous administrative account creations, and to monitor unauthorized modifications to critical system directories. This practical integration ensured that both deep host-level telemetry from the agents and broad external network data—including pfSense firewall logs, Snort IPS alerts, and Zeek protocol metadata—were successfully unified and analyzed within a single, cohesive analytical interface.

In conclusion, the successful theoretical mapping and practical, customized deployment of the Wazuh SIEM established the central nervous system of the SOC. By achieving total visibility

across the simulated enterprise environment, the platform provided the foundational intelligence required to support the subsequent integration of active perimeter defenses and automated orchestration workflows.

# 2. Perimeter Defense: Active Intrusion Prevention and Passive Network Monitoring

While gaining deep visibility into endpoint activity via a SIEM is critical, the ability to actively detect and neutralize threats as they traverse the network boundary, combined with passive analysis for deep protocol visibility, remains an essential component of defense-in-depth. This section discusses the theoretical distinctions between active inline prevention and passive network monitoring, followed by the practical implementation of both approaches within the GM-Soft architecture.

From a theoretical standpoint, network security monitoring can be approached through two complementary paradigms. Active inline prevention operates directly within the network data path, inspecting every packet in real-time and possessing the authority to drop, reject, or reset connections that match malicious signatures before they can penetrate the internal network. This approach requires careful, continuous tuning, as overly aggressive configurations can lead to false positives that inadvertently block legitimate business traffic. Conversely, passive network monitoring operates out-of-band, analyzing copies of network traffic without interfering with the actual data flow. While it cannot block threats directly, it provides rich, protocol-level metadata that enables deep forensic analysis and detection of anomalous behaviors that signature-based systems might miss.

Practically, the active inline prevention layer was realized for GM-Soft by deploying pfSense, a highly robust, enterprise-grade routing platform based on the FreeBSD operating system, integrated with the Snort IPS module. The pfSense virtual appliance was strategically positioned at the absolute edge of the Azure virtual network. It was configured with highly restricted, dual virtual interfaces—representing a Wide Area Network (WAN) facing the simulated internet and a Local Area Network (LAN) protecting the internal servers—to govern traffic flow securely. Once the foundational routing policies and basic packet-filtering firewall rules were established, the Snort package was installed directly via the pfSense package manager. Crucially, the Snort interface was configured to operate in inline IPS mode on the external WAN interface, granting it the capability to actively sever malicious connections in real-time.

To complement this active inline protection, a Zeek sensor was deployed within a Docker container on the orchestration VM. Zeek operates in a purely passive mode, analyzing copies of network traffic without interfering with the actual data flow. Zeek excels at generating structured, metadata-rich logs for common application protocols—including HTTP (URLs, user-agents,

methods), DNS (queries, responses, types), SSL/TLS (certificates, cipher suites), SMB, and SMTP. These logs provide deep contextual information about network activities that signature-based systems might miss, such as DNS tunneling, algorithmically generated domain (DGA) connections, or suspicious file transfers. All Zeek-generated logs were forwarded via syslog to the Wazuh manager for correlation with other security events and to the secondary ELK Stack for long-term retention and forensic analysis.

To ensure the accuracy and relevance of the detection engine, extensive community-driven signature sets were required for Snort. Specifically, the Emerging Threats Open ruleset and the Snort Community ruleset were activated, downloaded, and compiled into the engine's memory. A vital, final integration step involved establishing a communication bridge between this perimeter defense and the central nervous system. The pfSense firewall was configured to forward its raw traffic routing logs, as well as the specific Snort detection alerts, via remote syslog directly to the previously established Wazuh manager. Similarly, Zeek logs were ingested into the SIEM for correlation with other security events. This integration allowed the central SIEM to correlate external, broad-spectrum network scanning attempts with localized internal endpoint behaviors, providing a holistic view of an unfolding attack.

By transitioning from theoretical concepts of network traffic analysis to the concrete deployment of a fully inline Snort IPS and a passive Zeek sensor, the architecture successfully established a formidable perimeter monitoring and defense barrier. This combination proved capable of both localized, immediate threat blocking and centralized, correlated reporting to the broader SOC ecosystem.

# 3. Intelligence Integration, Incident Response, and SOAR Orchestration

The final architectural layer of the SOC is arguably the most transformative, elevating the platform from a passive monitoring system reliant on human intervention into a proactive, self-healing, machine-speed ecosystem. This section explores the advanced theories of threat intelligence sharing, incident management, and security orchestration, followed by an explanation of their complex programmatic integration.

Theoretically, the operational efficiency and accuracy of a SOC are heavily reliant on the quality and timeliness of its threat intelligence. Threat Intelligence platforms exist to aggregate and normalize Indicators of Compromise (IoCs)—such as recently identified malicious IP addresses, known phishing domains, or cryptographic hashes of novel malware strains—sourced from global cybersecurity communities and governmental sharing initiatives. Concurrently, Security Orchestration, Automation, and Response (SOAR) platforms address the most critical human issue in modern cybersecurity: "alert fatigue." Analysts are frequently overwhelmed

by thousands of daily SIEM alerts. By relying on predefined, logic-based digital playbooks, a SOAR platform can automatically intercept a SIEM alert the moment it is generated, programmatically query external intelligence databases to enrich the context of the alert, and directly interact via APIs with firewalls or identity management systems to contain the threat. This process occurs in milliseconds, entirely eliminating human delay during the critical initial phases of incident response.

Within the practical laboratory environment, these theories were realized through the deployment of the Malware Information Sharing Platform (MISP) and a suite of containerized services running on the Shuffle-Orchestrator VM. MISP was installed on a dedicated server and configured to actively subscribe to highly reputable global intelligence feeds provided by the AlienVault Open Threat Exchange and the CIRCL OSINT feeds. However, simply possessing intelligence is insufficient; it must be operationalized. To achieve this, a custom Python script was engineered and scheduled via Linux cron jobs to execute autonomously every day. This script programmatically queried the MISP REST API, extracted the newly published IoCs, translated them into Wazuh-compatible XML detection rules, injected them directly into the SIEM's configuration directory, and forced a silent restart of the monitoring engine to activate the new defenses.

Simultaneously, the Shuffle orchestrator was deployed leveraging Docker containerization to ensure environmental stability. A sophisticated, multi-stage response workflow was graphically designed and coded within the Shuffle interface. The workflow functioned as follows: when the Wazuh SIEM detected an event breaching a critical severity threshold (level 12 or higher), it instantly dispatched a JSON-formatted webhook trigger to Shuffle. The orchestrator was programmed to parse this payload to extract the attacking source IP address. It then queried the external VirusTotal API to ascertain the global reputation score of that IP. Upon confirming malicious intent—defined as three or more independent vendor detections on VirusTotal—Shuffle utilized an authenticated REST API request to directly command the pfSense firewall to instantly instantiate a dynamic block rule against the attacker. Finally, the workflow generated and dispatched a comprehensive, context-enriched notification email to the human SOC analyst, detailing the alert and confirming that containment actions had already been executed.

To support structured incident handling, TheHive was deployed within the same containerized environment as the central case management platform. TheHive enables collaborative case tracking with features including task assignment, timeline management, and evidence preservation. Whenever a critical alert is generated, it can be escalated to a new case in TheHive, ensuring that every incident is systematically tracked from detection through resolution. Complementing TheHive, Cortex provides automated observables analysis through its extensive library of analyzers and responders. When an IoC is identified, Cortex automatically queries multiple threat intelligence sources (including VirusTotal, MISP, and other external feeds) to enrich the context and severity of the alert. This enrichment is then fed back into the case

management workflow, providing SOC analysts with comprehensive threat intelligence at their fingertips.

Additionally, a secondary ELK Stack was deployed on the orchestration VM to serve as a dedicated forensic log repository. This stack ingests historical logs from all sources—including Wazuh, pfSense, Snort, Zeek, and system logs—providing long-term retention for post-incident investigations and compliance auditing. The ELK Stack's powerful visualization capabilities enable deep-dive analysis of security events, trend identification, and proactive threat hunting activities.

Through the successful, programmatic marriage of MISP's dynamic intelligence feeds, Shuffle's automated response playbooks, TheHive's structured case management, Cortex's enrichment capabilities, and the secondary ELK Stack's forensic retention, the SOC platform transcended basic alerting. It successfully achieved the theoretical ideal of machine-speed incident response, closing the loop between detection, analysis, and containment without requiring manual human authorization.

## Conclusion

Chapter 2 has thoroughly addressed the theoretical concepts underpinning SIEM architectures, active intrusion prevention (Snort IPS on pfSense), passive network monitoring (Zeek), threat intelligence (MISP), security orchestration (Shuffle), incident management (TheHive), observables enrichment (Cortex), and forensic log retention (secondary ELK Stack), whilst immediately detailing their practical, rigorous execution within the GM-Soft cloud environment. This combined pedagogical and engineering approach successfully yielded a fully integrated, defense-in-depth architecture capable of autonomous operation. However, an architecture is only as strong as its performance under duress. The final chapter will therefore proceed to empirically validate this newly deployed architecture by subjecting it to a series of highly realistic, simulated cyberattack scenarios.

# Chapter 3

# Validation, Testing, and Operational Results

## Introduction

With the theoretical framework thoroughly established and the diverse technical components successfully integrated into a cohesive architectural pipeline, it was absolutely imperative to subject the platform to rigorous empirical validation. A security system cannot be deemed operational until its defensive capabilities are tested under realistic adversarial conditions. This final chapter details the extensive testing phase of the internship. It systematically outlines the diverse simulated attack scenarios executed against the cloud infrastructure, documents the system's automated detection and containment responses, provides a critical, quantitative evaluation of the platform's overall performance metrics, and presents the results of the LLM-powered reporting system and OpenSearch Anomaly Detection plugin.

## 1. Simulated Attack Scenarios

To accurately and safely assess the detection fidelity and automated containment capabilities of the newly deployed SOC, a dedicated attacker machine was provisioned on an isolated subnet within the Azure environment. Utilizing Kali Linux—the industry standard for penetration testing—four distinct, highly realistic cyberattack scenarios were executed against the monitored target endpoints. These scenarios were specifically designed to test different layers of the defense-in-depth architecture.

### 1.1 Network Reconnaissance (Nmap Scan)

The first validation test involved an aggressive network reconnaissance phase utilizing advanced port scanning utilities, specifically Nmap. This scenario illustrates the critical, omnipresent risk of external adversaries actively enumerating exposed services to identify vulnerable attack vectors. The attacker machine executed a comprehensive, high-speed SYN sweep across all ports on the target server. Immediately, the inline Snort IPS recognized the distinct packet signature of the aggressive scan. Simultaneously, the pfSense firewall logged the massive spike

in connection attempts and forwarded this abnormal traffic pattern data to the Wazuh SIEM. The SIEM rapidly correlated the firewall events with the IPS alerts, triggering a reconnaissance warning on the central dashboard. This provided the simulated SOC analyst with vital early warning visibility before any actual exploitation could occur.

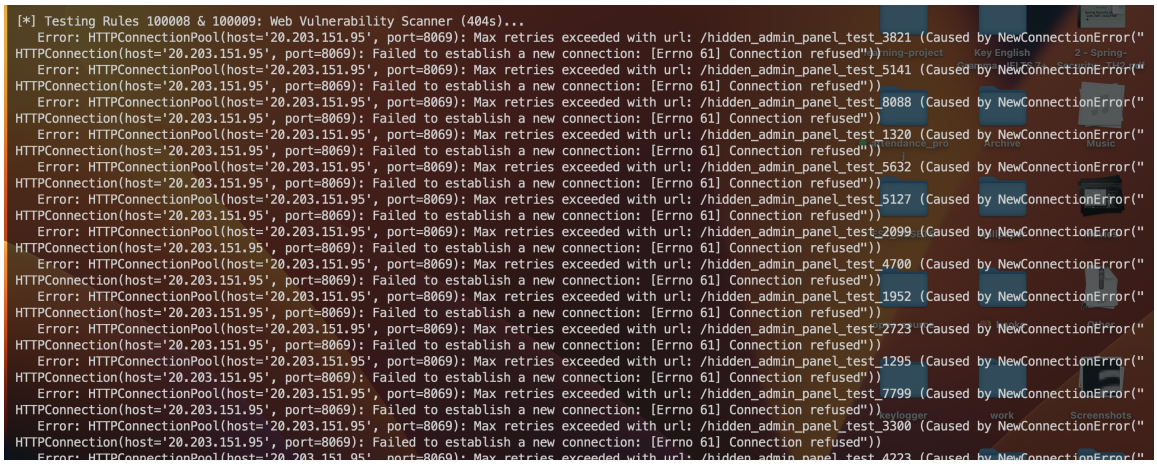



**Figure 3.1:** Performing a full vulnerability and port scan against the target system.

## 1.2 SSH Brute-Force Attack

The second scenario escalated the threat profile significantly by simulating a direct, aggressive brute-force authentication attack against a Linux endpoint's Secure Shell (SSH) service. Using the automated password-guessing tool Hydra, the simulated attacker generated dozens of rapid, failed login attempts utilizing a dictionary of common passwords. Because the attack targeted an encrypted protocol, the network IPS could not inspect the payload. However, the Wazuh security agent installed deep within the target machine captured the systemic authentication failures from the local authentication logs and streamed them instantly to the central manager. The custom correlation rules formulated during the deployment phase successfully identified the velocity of the failures, breached the designated threshold, and elevated the alert to a critical severity. This critical classification instantly triggered the webhook to the SOAR orchestrator, which verified the hostile intent and commanded the firewall to permanently sever the attacker's connection.

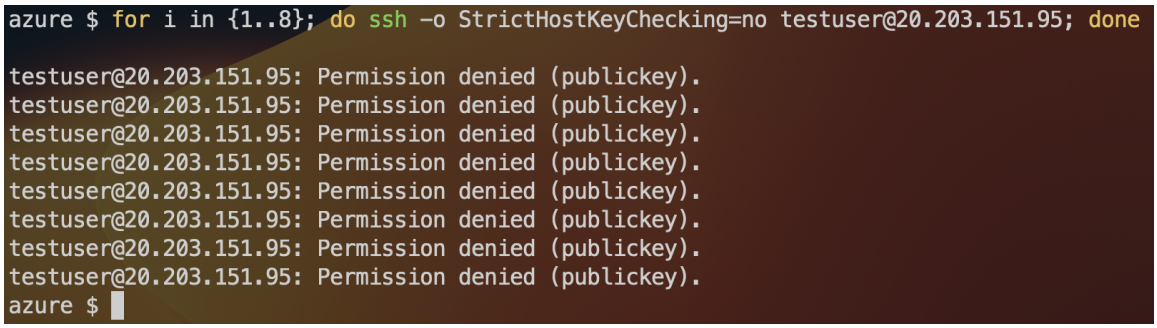



**Figure 3.2:** Simulating a high-velocity SSH brute-force attack against the target VM using invalid credentials.

| | | | |
|---|---|---|---|
| | Jun 17, 2026 @ 16:51:37.324 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:37.321 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:35.323 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:35.320 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:33.360 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:33.360 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:51:33.319 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:50:05.343 | odoo_agent | sshd: Attempt to login using a non-existent user |
| | Jun 17, 2026 @ 16:50:05.302 | odoo_agent | sshd: Attempt to login using a non-existent user |

**Figure 3.3:** Real-time detection of repeated SSH authentication failures originating from the attacker node.

## 1.3 Malware Deployment via Path Traversal

The third scenario tested the platform's ultimate, host-level defense mechanisms by simulating a successful web application exploit leading to the deployment of a malicious payload. Following a simulated path traversal attack—a technique allowing adversaries to access restricted directories outside the intended web root—a known malware test file was covertly downloaded and written to a sensitive directory on the target server. The Wazuh File Integrity Monitoring (FIM) module, which continuously scans critical directories for unauthorized changes, instantly detected the anomalous file creation. It calculated the cryptographic hash of the new file, matched it against known threat signatures, and generated an immediate critical alert. This proved the system's vital capability to detect post-exploitation activities and malware deployment deep within the host filesystem, even if the initial network perimeter was compromised.

```
[*] Testing Rule 100010: Path Traversal / Command Injection...
   Error: HTTPConnectionPool(host='20.203.151.95', port=8069): Max
t='20.203.151.95', port=8069): Failed to establish a new connectio
   Error: HTTPConnectionPool(host='20.203.151.95', port=8069): Max
nection(host='20.203.151.95', port=8069): Failed to establish a ne
   Error: HTTPConnectionPool(host='20.203.151.95', port=8069): Max
ection(host='20.203.151.95', port=8069): Failed to establish a new
[-] Injection test complete.
```

**Figure 3.4:** File Integrity Monitoring alert triggered by the simulated malicious payload deployment.

## 1.4 Odoo ERP Attack Simulation

The fourth scenario specifically targeted the Odoo ERP application, validating the custom Wazuh rules (IDs: 100005 and 100006) designed to detect brute-force login attempts and path traversal exploits. The attacker machine executed a Python script using the ‘requests‘ library to generate rapid authentication attempts against the Odoo web login endpoint (‘/web/login‘), cycling through a dictionary of common passwords.

```
intissar@DESKTOP-7IP6PIV:/mnt/c/Users/HP/Downloads/soc-tests$ bash attack_test1.sh
Starting simulated attacks against http://20.203.151.95:8069...

[*] Testing Rules 100005 & 100006: Odoo Brute Force...
   Attempt 1: POST /web/login - Status: 400
   Attempt 2: POST /web/login - Status: 400
   Attempt 3: POST /web/login - Status: 400
   Attempt 4: POST /web/login - Status: 400
   Attempt 5: POST /web/login - Status: 400
   Attempt 6: POST /web/login - Status: 400
   Attempt 7: POST /web/login - Status: 400
   Attempt 8: POST /web/login - Status: 400
```

**Figure 3.5:** Odoo brute force attack simulation showing HTTP 400 responses from the target.

As shown in Figure 3.5, each attempt returned HTTP 400 (Bad Request), confirming that the Odoo server correctly rejected invalid login credentials. The repeated failures were captured by the Wazuh agent on the Odoo server, triggering the custom correlation rules which identified the pattern (5+ failures within 60 seconds). The alerts were then forwarded to the OpenSearch Anomaly Detection plugin and the Shuffle SOAR engine, which initiated automated containment actions via the pfSense firewall.

These four highly diverse scenarios successfully validated that the system was exceptionally capable of detecting and responding to threats across multiple layers of the OSI model, ranging from initial perimeter reconnaissance scanning to deep, host-level filesystem compromise and application-layer attacks.

# 2. Performance Evaluation and Operational Insights

Following the successful execution and documentation of the simulated attack scenarios, a rigorous quantitative evaluation of the system's performance was conducted. This evaluation was necessary to determine if the ambitious performance objectives set at the project's inception were met, and to provide an honest analysis of areas requiring ongoing operational refinement by the GM-Soft engineering team.

The automated response metrics observed during the testing phase were exceptionally positive, exceeding initial expectations. Across all critical attack scenarios, the Mean Time to Detect (MTTD)—measured precisely from the exact initiation of the malicious network activity to the generation of a normalized, prioritized alert on the SIEM dashboard—consistently remained under three seconds. More importantly, the Mean Time to Respond (MTTR)—which encompassed the entire automated workflow including the SIEM webhook dispatch, the SOAR engine's external reputation enrichment query via VirusTotal, and the programmatic execution of the pfSense firewall block rule—was reliably clocked at under five seconds. This represents a massive, paradigm-shifting operational improvement over traditional manual intervention methods. The system effectively neutralized the simulated threats before human analysts were even required to open their notification emails, drastically reducing the organization's theoretical risk exposure.

**Table 3.1:** Empirical performance metrics recorded across all attack simulations

| Metric Category | Observed Result |
|---|---|
| Mean Time to Detect (MTTD) | $< 3$ seconds |
| Mean Time to Respond (MTTR) | $< 5$ seconds |
| Initial False Positive Rate | $\approx 8\%$ (pre-tuning) |
| Post-Tuning False Positive Rate | $\approx 3\%$ |
| Total Software Licensing Cost | \$0 (Exclusively Open-Source) |
| LLM Report Generation | Daily automated report via cron job (24-hour cycle) |
| Anomaly Detection Sensitivity | Detected 5 statistical deviations (anomaly grade: 1.00) |

*Source: Empirical laboratory testing results, GM-Soft (2026).*

However, the evaluation phase also highlighted necessary operational realities regarding the long-term maintenance of the system. Initially, the activation of the extensive Emerging Threats network intrusion rulesets resulted in a false positive rate of approximately eight percent. These false alerts were mostly triggered by unconventional but entirely benign internal routing protocols and legitimate administrative scripts. This necessitated a rigorous, time-consuming tuning phase, where specific suppression rules had to be carefully crafted to quiet noisy alerts without blinding the sensors to genuine threats. Through systematic tuning, this false positive rate was

eventually reduced to a highly manageable three percent.

Additionally, it was noted that the threat intelligence synchronization script currently operates on a daily execution schedule. While efficient, this introduces a slight latency window—up to twenty-four hours—between the global publication of a new threat indicator and its active detection capability within the local SIEM. Addressing this latency will be a priority for future iterations of the platform.

Ultimately, the quantitative results and operational insights conclusively demonstrated that an enterprise-grade, highly responsive security architecture can be successfully implemented utilizing exclusively open-source technologies. The platform met all functional requirements, provided that sufficient ongoing engineering effort is dedicated to continuous integration, rule tuning, and playbook refinement.

# 3. LLM-Generated Report Results

Beyond automated detection and containment, the SOC platform incorporates a Large Language Model (LLaMA 3.3-70B) accessed via the Groq API to generate comprehensive security reports. These reports are automatically produced every 24 hours and dispatched to the SOC analyst's email, providing a structured overview of the security posture, attack classifications, and actionable remediation recommendations.

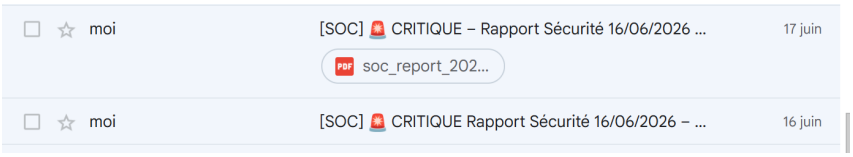



**Figure 3.6:** Email notification with the automatically generated PDF report attached.

As shown in Figure 3.6, the SOC analyst receives an email notification with the subject "[SOC] CRITIQUE – Rapport Sécurité 16/06/2026", indicating the severity level and date of the report. The email includes the automatically generated PDF report as an attachment, ready for review and distribution.




**Figure 3.7:** Cover page of the LLM-generated SOC report.

Figure 3.7 displays the cover page of the generated PDF report. The cover includes the organization name ("SOC Wazuh – ERP Odoo"), the analysis period (24 hours from June 16 at 08:30 UTC to June 17 at 08:30 UTC), and the generation timestamp (June 17 at 08:30 UTC). This structured format ensures that SOC analysts can quickly identify the report's scope and relevance.

The generated report provides the following key sections:

- **Executive Summary:** A high-level overview including total alerts by severity level, detected anomalies, and top threats. For the analyzed period, the LLM identified 303 total alerts (20 high, 6 medium, 277 low), with Docker error messages (148 alerts) and SSH non-existent user login attempts (75 alerts) as the dominant threats. MITRE tactics such as Lateral Movement, Credential Access, and Defense Evasion were flagged, leading to an overall risk assessment of "ÉLEVÉ" (High).
- **Top 3 Immediate Actions:** The LLM recommended: (1) hardening Docker configuration, (2) enabling two-factor authentication for SSH connections, and (3) reviewing and updating Odoo password policies. The system strictly excludes IP-blocking suggestions as this functionality is already handled automatically by the Azure Security Group and Shuffle SOAR orchestration.
- **Detailed Rule Analysis:** For each critical and high-severity alert, the LLM generates a structured analysis including: attack type classification, MITRE technique mapping (e.g., T1110 - Brute Force), risk score (8/10), business impact assessment for Odoo, immediate remediation steps (excluding IP blocking), and long-term prevention recommendations

for both Wazuh and Odoo configurations.

- **Visual Dashboards:** The report includes charts depicting alert timelines, severity distribution, top rules triggered, top source IP addresses, and MITRE tactic frequencies, providing a comprehensive visual overview of the security landscape.

The LLM-generated report transforms raw security alerts into actionable intelligence, significantly reducing the cognitive load on human analysts and accelerating incident response decision-making. This automated reporting capability ensures that SOC analysts receive a daily, structured security briefing without manual intervention.

# 4. OpenSearch Anomaly Detection Results

The OpenSearch Anomaly Detection plugin was configured to monitor Wazuh alerts generated by custom rules (IDs: 100005, 100006, 100008, 100009, 100010) corresponding to brute force attempts, path traversal exploits, and web scanning activities. The detector, named "odoo-ssh-attack-detector," operates on one-minute intervals, counting events and identifying statistical deviations from established behavioral baselines.

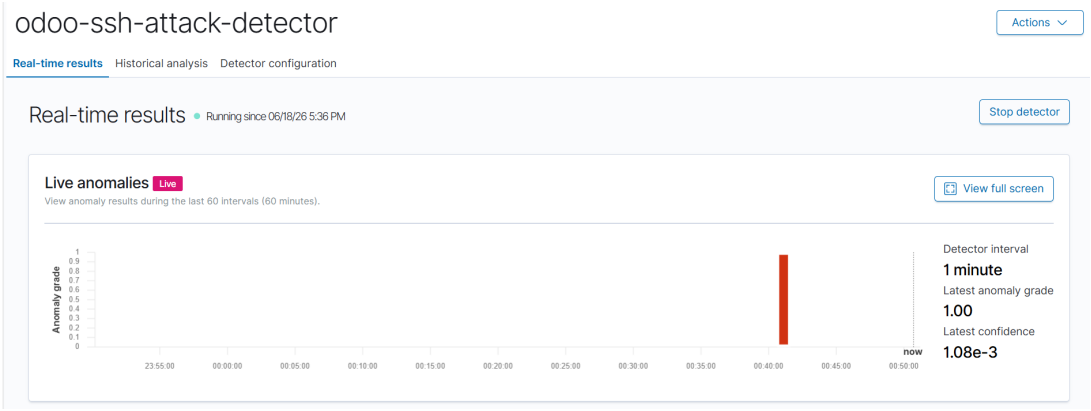



**Figure 3.8:** Real-time anomaly detection dashboard showing live anomaly grade (1.00) and confidence (1.08e-3).

Figure 3.8 presents the real-time anomaly detection dashboard. The detector has been running since June 18, 2026 at 5:36 PM. The graph displays anomaly grades over the last 60 intervals (60 minutes), with the latest anomaly grade reaching 1.00 (the maximum value), indicating a significant statistical deviation from the established baseline. The confidence score of 1.08e-3 confirms the reliability of this detection. This real-time monitoring capability enables the SOC to identify ongoing attacks as they occur, even when they operate below traditional signature-based thresholds.

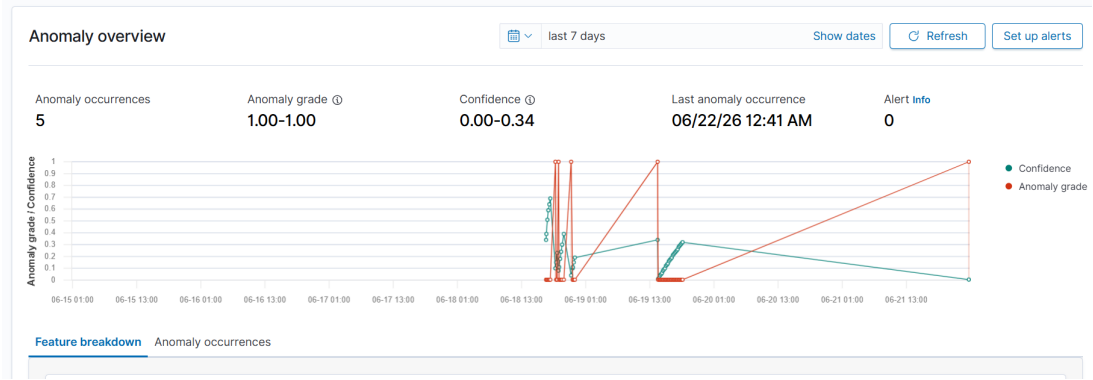



**Figure 3.9:** Historical anomaly overview showing 5 anomaly occurrences with grades between 1.00 and 1.00.

Figure 3.9 displays the historical anomaly overview for the detector. The system identified 5 anomaly occurrences over the monitored period (June 15–22, 2026), each with an anomaly grade of 1.00 (maximum). The last anomaly occurred on June 22 at 12:41 AM. The chart shows both the anomaly grade (blue line) and confidence level (red line) over time, demonstrating consistent high-confidence anomaly detection during attack periods. This historical view enables SOC analysts to identify recurring attack patterns and correlate them with other security events.

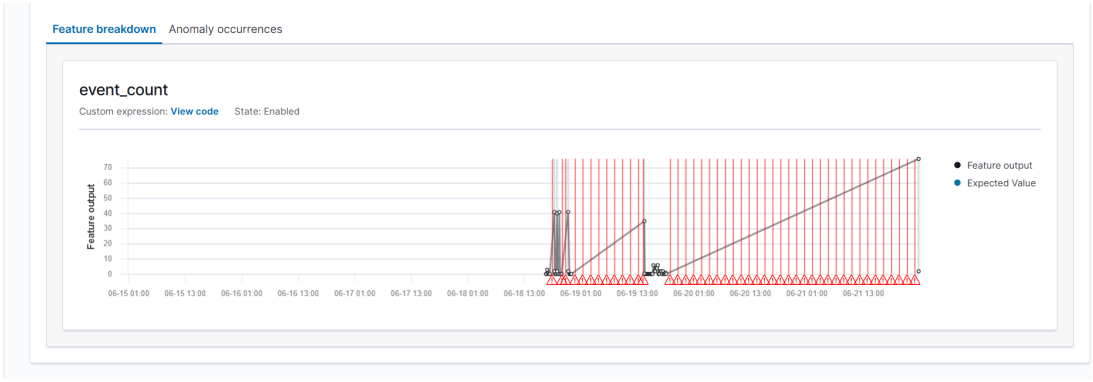



**Figure 3.10:** Feature breakdown showing event$_c ountanomaliesovertimewithexpectedvalues$.

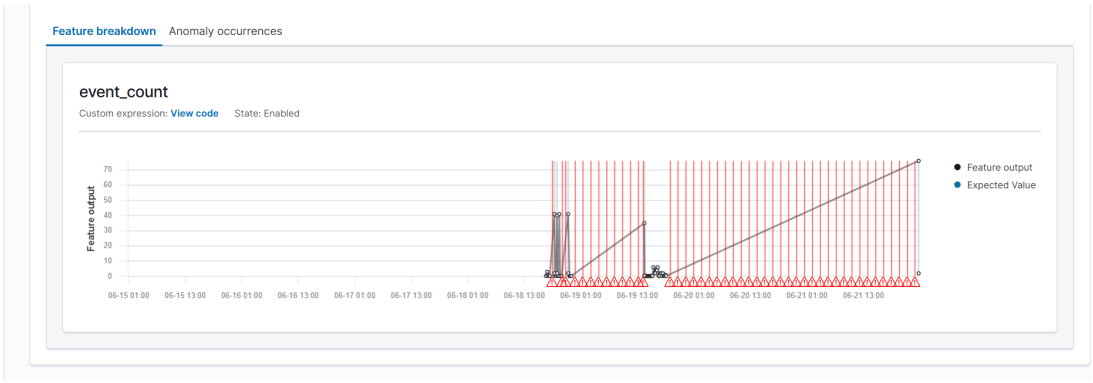



**Figure 3.11:** Feature breakdown showing event$_c ountanomaliesovertimewithexpectedvalues$.

Figure 3.11 provides a detailed breakdown of the event count feature, which counts the number of events matching the monitored rule IDs. The visualization compares the actual feature output (blue bars) against the expected value (red line), highlighting periods where actual activity significantly exceeded the predicted baseline. This statistical approach enables detection of subtle attack patterns—such as low-and-slow brute force attempts or intermittent scanning activities—that might evade static rule-based correlation.

The Anomaly Detection plugin successfully identified:

- **5 high-confidence anomalies** during the monitoring period, each with anomaly grade 1.00
- **Real-time detection** with anomaly grades reaching maximum within one-minute intervals
- **Statistical deviations** in event frequencies that correlated with Odoo brute force attempts and SSH scanning activities

These results validate that the anomaly detection capability effectively complements signature-based detection, providing an additional layer of defense against zero-day attacks and sophisticated threat actors that operate below traditional detection thresholds.

# Conclusion

Chapter 3 successfully validated the theoretical designs and practical implementations constructed in the earlier chapters. By subjecting the infrastructure to rigorous, multi-layered simulated attacks, the platform definitively proved its capability to autonomously detect and neutralize complex threats within seconds.

The integration of the LLM-powered reporting system further enhanced the SOC's operational efficiency by automatically generating comprehensive security advisories with attack classifications, business impact assessments, and remediation recommendations—all delivered daily via email without manual intervention. The OpenSearch Anomaly Detection plugin successfully identified 5 statistical deviations in event patterns with maximum anomaly grades of 1.00, demonstrating its effectiveness in detecting attacks that operate below signature-based thresholds. The attack simulation validated the correct triggering of custom rules (100005 and 100006) and the successful correlation between Wazuh detection, OpenSearch anomaly identification, and Shuffle SOAR automated response.

While the testing phase highlighted the ongoing necessity of vigilant rule tuning to manage false positives, the empirical results confirm that the core objectives of the internship were unequivocally achieved. The project has yielded a highly functional, production-ready SOC prototype that significantly enhances GM-Soft's defensive posture.

# General Conclusion

The internship at GM-Soft proved to be an exceptionally formative and challenging experience, providing a unique and highly valuable opportunity to bridge the gap between academic cybersecurity theory and rigorous, practical cloud systems engineering. The core ambition of this project was to directly address the organization's vulnerabilities—specifically decentralized logging mechanisms and dangerously slow manual incident response workflows—by designing, engineering, and implementing a fully functional Security Operations Center prototype from the ground up.

Through the disciplined application of Agile methodologies across a structured ten-week period, the project successfully delivered a robust, defense-in-depth architecture. By exclusively leveraging powerful, community-driven open-source tools—namely Wazuh, the ELK stack, pfSense, Snort, Zeek, MISP, Shuffle, TheHive, Cortex, OpenSearch Anomaly Detection, and the Groq LLM API—the delivered solution achieved true enterprise-grade visibility without incurring the prohibitive, recurring licensing costs usually associated with commercial security platforms. The deployed system proved highly capable of centralizing massive volumes of log data from heterogeneous operating systems and network boundaries, applying intelligent correlation rules to identify genuine threats amidst the noise of standard network traffic.

Most notably, the successful programmatic integration of security orchestration, AI-powered analytics, and global threat intelligence transformed the platform from a passive monitoring dashboard into a highly active, automated defense mechanism. As empirically validated through rigorous attack simulations involving port scanning, brute-force authentication attempts, malware deployment, and Odoo-specific web attacks, the orchestrated SOAR workflows successfully achieved a mean time to respond of under five seconds. This automated containment capability drastically minimizes the organization's risk exposure window during an active cyberattack, fulfilling the most critical and ambitious objective defined at the outset of the project.

The incorporation of the Groq API (LLaMA 3.3-70B) for automated report generation and the OpenSearch Anomaly Detection plugin for statistical deviation analysis have proven to be valuable additions to the SOC architecture. The LLM-powered reporting system successfully transformed raw security alerts into actionable intelligence with attack classifications, business impact assessments, and remediation recommendations, significantly reducing the cognitive load on human analysts. The anomaly detection plugin effectively identified 5 high-confidence statistical deviations that correlated with Odoo brute force attempts and SSH scanning activities, demonstrating the value of machine learning-based detection in complementing traditional

signature-based rules. These innovations have strengthened the SOC's proactive detection posture and enriched the automated decision-making context for the SOAR orchestration layer.

# Perspectives

Looking toward the future operationalization and enhancement of this prototype within GM-Soft's production environment, several strategic recommendations and perspectives are proposed:

1. **Real-Time Threat Intelligence Synchronization:** Transitioning the daily threat intelligence synchronization script to a real-time event-streaming model via message brokers (e.g., Apache Kafka or RabbitMQ) would eliminate current latency windows regarding newly discovered Indicators of Compromise, enabling near-instantaneous defense updates.
2. **MITRE ATT&CK Playbook Standardization:** Dedicating resources to developing a comprehensive, standardized library of automated response playbooks aligned specifically with the MITRE ATT&CK framework would provide the SOC with predictable, audited responses to a much wider array of advanced persistent threats. This would also facilitate compliance with industry security standards.
3. **Stabilization of TheHive and Cortex Integration:** As the incident-response layer (TheHive, Cortex, and the secondary ELK Stack) is currently in an active debugging state, prioritizing the resolution of authentication and API integration issues will complete the SOC's defense-in-depth by providing structured case management, automated artifact enrichment, and deep historical log analysis.
4. **LLM Prompt Refinement and Model Fine-Tuning:** Continuously refining the prompt engineering strategy for the Groq API and exploring fine-tuning options for the LLaMA model on security-specific datasets would further improve classification accuracy and reduce hallucination risks in generated reports.
5. **Production Scaling and Client Offering:** The current prototype, built on Microsoft Azure, provides a highly scalable and cost-effective model. GM-Soft is well-positioned to package this open-source SOC solution as a managed security service offering for its SME clients in the Moroccan and European markets, generating a new revenue stream while enhancing the cybersecurity posture of its broader ecosystem.

Ultimately, this engineering project has not only fortified GM-Soft's internal security posture but has also successfully laid a highly scalable, economically viable foundation for their future cybersecurity service offerings in the European and African markets.

# Bibliography


1. Chuvakin, A., & H., K. (2016). *How to build a Security Operations Center (SOC)*. SANS Institute.

2. GM-Soft. (2026). *Company presentation and project specifications* [Internal documents]. GM-Soft.

3. Elastic N.V. (2023). *Elastic Stack documentation*. Retrieved from https://www.elastic.co/guide/

4. MISP Project. (2023). *MISP user guide*. Retrieved from https://www.misp-project.org/documentation/

5. Netgate. (2023). *pfSense documentation*. Retrieved from https://docs.netgate.com/

6. Shuffle. (2023). *Shuffle documentation*. Retrieved from https://shuffler.io/docs

7. Snort.org. (2023). *Snort users manual*. Cisco Systems.

8. Wazuh, Inc. (2023). *Wazuh documentation*. Retrieved from https://documentation.wazuh.com/

9. Cortex Project. (2024). *Cortex user guide*. Retrieved from https://cortex-project.org/documentation

10. Groq Inc. (2024). *Groq API documentation*. Retrieved from https://console.groq.com/docs

11. OpenSearch Project. (2024). *Anomaly Detection plugin documentation*. Retrieved from https://opensearch.org/docs/latest/monitoring-plugins/ad/

12. TheHive Project. (2024). *TheHive documentation*. Retrieved from https://thehive-project.org/documentation

13. Zeek Project. (2024). *Zeek user manual*. Retrieved from https://docs.zeek.org/

# List of Appendices

1. Appendix 1: Wazuh and ELK Stack — Detailed Installation Commands . . . . . 34
2. Appendix 2: OpenSearch Anomaly Detection — Detector Configuration Script . . . . . 35
3. Appendix 3: LLM-Powered SOC Report Generator . . . . . 41

# Appendix 1: Wazuh + ELK: Detailed Installation Steps

This appendix provides the exact, chronological command-line sequences utilized by the engineering team to deploy the core SIEM infrastructure on the Microsoft Azure Ubuntu 20.04 virtual machine. Prior to executing these installation scripts, the environment required a strict baseline configuration. This included provisioning a minimum of four gigabytes of random-access memory, two virtual central processing units, and fifty gigabytes of solid-state storage. Furthermore, a static IP configuration was mandated, alongside precise cloud network security group rules permitting bidirectional traffic on vital monitoring ports, specifically TCP ports 1514 and 1515 for agent communication, and HTTPS port 443 for the dashboard interface.

Once the prerequisite environmental conditions were satisfied, the automated installation sequence was executed sequentially as follows to provision the indexer, the central manager, and the dashboard visualization tools:

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
curl -sO https://packages.wazuh.com/4.7/config.yml
# Administrative intervention to define cluster node IP addresses
nano config.yml
bash wazuh-install.sh --generate-config-files
bash wazuh-install.sh --wazuh-indexer node-1
bash wazuh-install.sh --start-cluster
bash wazuh-install.sh --wazuh-server wazuh-1
bash wazuh-install.sh --wazuh-dashboard dashboard
```

**Listing 1:** Complete Wazuh architectural installation sequence

# Appendix 2: OpenSearch Anomaly Detection - Detector Configuration Script

This appendix provides the Python implementation used to configure and deploy the OpenSearch Anomaly Detection plugin for detecting web application attacks targeting the Odoo ERP server. The script programmatically creates a detector that monitors Wazuh alerts generated by custom correlation rules (IDs: 100005, 100006, 100008, 100009, 100010) corresponding to brute force attempts, path traversal exploits, and web scanning activities. The detector operates on one-minute intervals, counting events and identifying statistical deviations from established behavioral baselines.

```python
import requests
import urllib3
import time
import sys

# Disable SSL warnings for self-signed certificates
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

#           Configuration


INDEXER_URL = "https://localhost:9200"
AUTH = ("admin", "SecretPassword")
DETECTOR_ID = "GMyK254BDomb_dRspaeN"

# Custom Wazuh rule IDs for Odoo web attacks
ODOO_RULE_IDS = ["100005", "100006", "100008", "100009", "100010"]

#           Functions



def stop_detector(detector_id):
    """Stops the detector if it is currently running."""
    stop_url = f"{INDEXER_URL}/_plugins/_anomaly_detection/detectors
```

```
        /{detector_id}/_stop"
    try:
        response = requests.post(stop_url, auth=AUTH, verify=False,
            timeout=30)
        if response.status_code in [200, 404]:
            print(f"[+] Detector {detector_id} stopped or not running
                .")
        else:
            print(f"[!] Stop response: {response.status_code} - {
                response.text}")
    except requests.exceptions.RequestException as e:
        print(f"[!] Error stopping detector: {e}")

def start_detector(detector_id):
    """Starts the detector."""
    start_url = f"{INDEXER_URL}/_plugins/_anomaly_detection/detectors
        /{detector_id}/_start"
    try:
        response = requests.post(start_url, auth=AUTH, verify=False,
            timeout=30)
        if response.status_code == 200:
            print(f"[+] Detector {detector_id} started successfully."
                )
        else:
            print(f"[!] Start failed: {response.status_code} - {
                response.text}")
        return response
    except requests.exceptions.RequestException as e:
        print(f"[!] Error starting detector: {e}")
        return None

def update_detector(detector_id):
    """Updates the detector configuration with filtering on Odoo rule
        IDs."""
    url = f"{INDEXER_URL}/_plugins/_anomaly_detection/detectors/{
        detector_id}"

    #           Detector Payload


    payload = {
```

```
        "name": "odoo-web-attack-detector",
        "description": "Anomaly detector for Odoo web attacks (brute force, path traversal, web scanning)",
        "time_field": "timestamp",
        "indices": ["wazuh-alerts-*"],
        "filter_query": {
            "bool": {
                "filter": [
                    {
                        "terms": {
                            "rule.id": ODOO_RULE_IDS
                        }
                    }
                ]
            }
        },
        "feature_attributes": [
            {
                "feature_id": "event_count",
                "feature_name": "event_count",
                "feature_enabled": True,
                "aggregation_query": {
                    "event_count": {
                        "value_count": {
                            "field": "_id"
                        }
                    }
                }
            }
        ],
        "detection_interval": {"period": {"interval": 1, "unit": "MINUTES"}},
        "window_delay": {"period": {"interval": 1, "unit": "MINUTES"}}
    }

    try:
        response = requests.put(url, auth=AUTH, json=payload, verify=False, timeout=30)
        if response.status_code == 200:
            print("[+] Detector updated successfully.")
```

```
            print(f"[*] Filtering on rule IDs: {', '.join(ODOO_RULE_IDS)}")
            return True
        else:
            print(f"[-] Update failed: {response.status_code} - {response.text}")
            return False
    except requests.exceptions.RequestException as e:
        print(f"[!] Error updating detector: {e}")
        return False

def get_detector_status(detector_id):
    """Retrieves and displays the current status of the detector."""
    status_url = f"{INDEXER_URL}/_plugins/_anomaly_detection/detectors/{detector_id}"
    try:
        response = requests.get(status_url, auth=AUTH, verify=False, timeout=30)
        if response.status_code == 200:
            data = response.json()
            status = data.get("detector", {}).get("status", "Unknown")
            print(f"[*] Detector status: {status}")
            return status
        else:
            print(f"[-] Status check failed: {response.status_code}")
            return None
    except requests.exceptions.RequestException as e:
        print(f"[!] Error checking status: {e}")
        return None

#           Main Execution



def main():
    print("=" * 60)
    print("  OpenSearch Anomaly Detection - Detector Configuration")
    print("=" * 60)
    print(f"[*] Detector ID: {DETECTOR_ID}")
    print(f"[*] Target rule IDs: {', '.join(ODOO_RULE_IDS)}")
```

```
    print()

    # Step 1: Stop existing detector
    print("[*] Step 1: Stopping existing detector...")
    stop_detector(DETECTOR_ID)
    time.sleep(2)

    # Step 2: Update configuration
    print("[*] Step 2: Updating detector configuration...")
    success = update_detector(DETECTOR_ID)
    time.sleep(2)

    if success:
        # Step 3: Start detector
        print("[*] Step 3: Starting detector...")
        start_detector(DETECTOR_ID)
        time.sleep(3)

        # Step 4: Verify status
        print("[*] Step 4: Verifying detector status...")
        status = get_detector_status(DETECTOR_ID)
        print()
        if status == "RUNNING":
            print("[+]     Detector is now RUNNING successfully.")
            print("[*] The detector will now analyze Odoo attack
               patterns in real-time.")
        else:
            print("[!]        Detector status is not RUNNING. Manual
               intervention may be required.")
    else:
        print("[!]     Detector update failed. Please check the
           configuration and try again.")
        sys.exit(1)

    print("=" * 60)

if __name__ == "__main__":
    main()
```

**Listing 2:** OpenSearch Anomaly Detection - Detector configuration and start script

This script performs the following operations:

1. **Stops** any existing detector instance to prevent conflicts.
2. **Updates** the detector configuration with a filter query targeting only Wazuh alerts generated by the custom Odoo attack rules (IDs 100005 through 100010).
3. **Starts** the detector with a one-minute detection interval and a one-minute window delay.
4. **Verifies** and displays the detector's operational status.

The detector counts events matching the specified rule IDs and applies unsupervised machine learning to identify statistical anomalies in the frequency of these events. This enables the detection of subtle attack patterns that might evade static rule-based correlation, such as low-and-slow brute force attempts or intermittent scanning activities.

# Appendix 3: LLM-Powered SOC Report Generator

This appendix provides the complete Python implementation of the automated SOC reporting system. The script integrates the Groq API (LLaMA 3.3-70B Versatile) with Wazuh/OpenSearch to generate comprehensive security reports, including executive summaries, rule-based attack analyses, and visual dashboards. The report is automatically dispatched via email to the SOC analyst on a daily schedule.

## Key Features

- **Alert Retrieval:** Fetches the last 24 hours of alerts from Wazuh via OpenSearch API.
- **Severity Classification:** Groups alerts by severity levels (CRITICAL → HIGH → MEDIUM → LOW).
- **Anomaly Detection:** Identifies statistical anomalies including temporal spikes, IP flooding, and brute-force bursts.
- **LLM Analysis:** Generates executive summaries and detailed rule analyses using Groq's LLaMA 3.3-70B model.
- **Visual Dashboards:** Creates charts for timeline, severity distribution, top rules, top IPs, and MITRE ATT&CK tactics.
- **PDF Export:** Converts HTML report to PDF using WeasyPrint.
- **Email Delivery:** Sends the report via SMTP (Gmail) with PDF attachment.

## Important Note

The system includes a strict rule preventing any recommendation to block IP addresses, as this functionality is already handled automatically by the Azure Security Group and Shuffle SOAR orchestration.

```
#!/usr/bin/env python3
import os, json, base64, io, smtplib, statistics, urllib3
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
```

```
from email import encoders
from datetime import datetime, timedelta, timezone
from collections import Counter, defaultdict

import requests
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
import markdown2
from groq import Groq
from dotenv import load_dotenv

load_dotenv()
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
# CONFIGURATION
ES_URL    = os.getenv("ES_URL",    "https://localhost:9200")
ES_USER   = os.getenv("ES_USER",   "admin")
ES_PASS   = os.getenv("ES_PASS",   "SecretPassword")
GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
GROQ_MODEL   = os.getenv("GROQ_MODEL",   "llama-3.3-70b-versatile")
SMTP_HOST   = os.getenv("SMTP_HOST",   "smtp.gmail.com")
SMTP_PORT   = int(os.getenv("SMTP_PORT", "587"))
SMTP_USER   = os.getenv("SMTP_USER",   "")
SMTP_PASS   = os.getenv("SMTP_PASS",   "")
EMAIL_FROM = os.getenv("EMAIL_FROM", SMTP_USER)
EMAIL_TO    = os.getenv("EMAIL_TO",    "soc-analyst@company.com")
ORG_NAME    = os.getenv("ORG_NAME",    "SOC Wazuh     ERP Odoo")
REPORTS_DIR = os.path.expanduser("~/soc-llm/reports")
os.makedirs(REPORTS_DIR, exist_ok=True)
os.makedirs(os.path.expanduser("~/soc-llm/logs"), exist_ok=True)
NOW    = datetime.now(timezone.utc)
START = NOW - timedelta(hours=24)
STAMP = NOW.strftime("%Y%m%d_%H%M")
HTML_FILE = os.path.join(REPORTS_DIR, f"soc_report_{STAMP}.html")
PDF_FILE  = os.path.join(REPORTS_DIR, f"soc_report_{STAMP}.pdf")
#


# PALETTES DE  S  V  R I T
#
```

```
SEV_MATRIX = [
    ("CRITIQUE", 12, 99, "#e74c3c", "bc", "      ", 4),
    ("  LEV  ",     9, 11, "#e67e22", "bh", "      ", 3),
    ("MOYEN",     6,  8, "#f39c12", "bm", "      ", 2),
    ("FAIBLE",    1,  5, "#27ae60", "bl", "      ", 1),
]

def level_to_sev(level: int) -> tuple:
    for label, lo, hi, color, css, emoji, prio in SEV_MATRIX:
        if lo <= level <= hi:
            return label, color, css, emoji, prio
    return "FAIBLE", "#27ae60", "bl", "      ", 1

def max_level_of(group: list) -> int:
    return max((a.get("rule", {}).get("level", 0) for a in group),
       default=0)

#


# 1. R CUP RATION DES ALERTES
#


def get_alerts(size: int = 2000) -> list:
    query = {
        "query": {"bool": {"filter": [
            {"range": {"@timestamp": {"gte": "now-24h", "lte": "now"
               }}}
        ]}},
        "sort": [
            {"rule.level":  {"order": "desc"}},
            {"@timestamp":  {"order": "desc"}},
        ],
        "size": size,
    }
    print("       R cup ration alertes Wazuh (24h     tri
       s  v  rit   d croissante)...")
    try:
```

```
        r = requests.get(
            f"{ES_URL}/wazuh-alerts-*/_search",
            auth=(ES_USER, ES_PASS),
            headers={"Content-Type": "application/json"},
            json=query, verify=False, timeout=30,
        )
        r.raise_for_status()
        alerts = [h["_source"] for h in r.json().get("hits", {}).get(
           "hits", [])]
        print(f"       {len(alerts)} alertes r cup r es")
        return alerts
    except Exception as e:
        print(f"       OpenSearch : {e}")
        return []

#


# 2. STATISTIQUES
#


def compute_stats(alerts: list) -> dict:
    sev_counts = {lbl: 0 for lbl, *_ in SEV_MATRIX}
    rules    = Counter()
    ips      = Counter()
    agents   = Counter()
    mitre_t  = Counter()
    mitre_id = Counter()
    hourly   = defaultdict(int)
    auth_fail = 0

    for a in alerts:
        lvl  = a.get("rule", {}).get("level", 0)
        desc = a.get("rule", {}).get("description", "Inconnu")
        src  = a.get("data", {}).get("srcip", "") or ""
        ag   = a.get("agent", {}).get("name", "unknown")

        label, *_ = level_to_sev(lvl)
        sev_counts[label] += 1
        rules[desc] += 1
```

```
        if src and src not in ("0.0.0.0", "N/A", ""):
            ips[src] += 1
        agents[ag] += 1

        mf = a.get("rule", {}).get("mitre", {})
        if isinstance(mf, dict):
            for t in mf.get("tactic", []): mitre_t[t] += 1
            for i in mf.get("id", []):     mitre_id[i] += 1
        elif isinstance(mf, list):
            for item in mf:
                if isinstance(item, str):
                    mitre_id[item] += 1
                elif isinstance(item, dict):
                    for t in item.get("tactic", []): mitre_t[t] += 1
                    if item.get("id"):               mitre_id[item["
                       id"]] += 1

        if any(k in desc.lower() for k in
               ("authentication fail", "invalid user", "brute force",
                "failed password", "echec", "invalid password")):
            auth_fail += 1

        ts = a.get("@timestamp", "")
        if ts and len(ts) >= 13:
            try:    hourly[int(ts[11:13])] += 1
            except: pass

    return {
        "total":        len(alerts),
        "severity":     sev_counts,
        "auth_fail":    auth_fail,
        "top_rules":    rules.most_common(10),
        "top_ips":      ips.most_common(10),
        "top_agents":   agents.most_common(5),
        "mitre_tactic": mitre_t.most_common(10),
        "mitre_id":     mitre_id.most_common(10),
        "hourly":       dict(sorted(hourly.items())),
    }
# 3. TRI S V RIT  + S LECTION
def group_and_sort(alerts: list) -> list[tuple]:
    grouped: dict[str, list] = defaultdict(list)
```

```
    for a in alerts:
        grouped[a.get("rule", {}).get("description", "Inconnu")].
            append(a)

    enriched = []
    for desc, group in grouped.items():
        mx = max_level_of(group)
        lbl, col, css, emoji, prio = level_to_sev(mx)
        enriched.append((desc, group, mx, lbl, col, emoji, css, prio)
            )

    enriched.sort(key=lambda x: (-x[7], -len(x[1])))
    return enriched

def select_for_llm(sorted_groups: list) -> list[tuple]:
    to_analyze, moyen_n = [], 0
    for item in sorted_groups:
        _, _, _, sev_lbl, *_ = item
        if sev_lbl in ("CRITIQUE", "  LEV  "):
            to_analyze.append(item)
        elif sev_lbl == "MOYEN" and moyen_n < 3:
            to_analyze.append(item)
            moyen_n += 1
    return to_analyze

#


# 4. ANOMALY DETECTION
#


def detect_anomalies(alerts: list, stats: dict) -> list[dict]:
    anomalies = []
    total = stats["total"]
    if not total: return anomalies

    hourly_vals = list(stats["hourly"].values())
    if len(hourly_vals) >= 4:
        mean = statistics.mean(hourly_vals)
        std  = statistics.stdev(hourly_vals) if len(hourly_vals) > 1
```

```
            else 0
        if std > 0:
            for hour, count in stats["hourly"].items():
                z = (count - mean) / std
                if z > 2.5:
                    anomalies.append({
                        "type":    "Pic Temporel Anormal", "icon": "
                            ", "sev": "CRITIQUE",
                        "detail":  f"Pic    {hour:02d}h00 UTC : {
                            count} alertes",
                        "recommendation": f"Investiguer les logs
                            entre {hour:02d}h00 et {hour+1:02d}h00 UTC
                            .",
                    })

    for ip, count in stats["top_ips"]:
        pct = count / total * 100
        if pct > 25:
            anomalies.append({
                "type":  "IP Flooding", "icon": "      ", "sev": "
                    CRITIQUE" if pct > 50 else "  LEV  ",
                "detail": f"IP {ip}     {pct:.1f}% des alertes",
                "recommendation": f"IP  d  j  transmise    Shuffle
                    pour blocage Azure. V rifier le Security Group.",
            })

    if stats["auth_fail"]:
        auth_pct = stats["auth_fail"] / total * 100
        if auth_pct > 40:
            anomalies.append({
                "type":  "Burst Brute-Force", "icon": "      ", "sev"
                    : "CRITIQUE",
                "detail": f"{stats['auth_fail']}  checs  auth ({
                    auth_pct:.1f}% du total)",
                "recommendation": "Activer le verrouillage de compte
                    Odoo et ajouter un CAPTCHA.",
            })

    anomalies.sort(key=lambda x: {"CRITIQUE": 0, "  LEV  ": 1, "MOYEN
        ": 2}.get(x["sev"], 3))
    return anomalies
```

```
218
219 #
220 # 5. GRAPHIQUES
221 #
222 _BG = "#1a1f35"; _DARKER = "#0a0e1a"; _BORDER = "#2d3561"; _FG = "#
    e0e0e0"
223 plt.rcParams.update({
224     "figure.facecolor": _BG, "axes.facecolor": _BG,
225     "text.color": _FG,         "axes.labelcolor": _FG,
226     "xtick.color": _FG,        "ytick.color": _FG,
227     "axes.edgecolor": _BORDER,"grid.color": _BORDER,
228     "axes.spines.top": False, "axes.spines.right": False,
229 })
230
231 def _b64(fig) -> str:
232     buf = io.BytesIO()
233     fig.savefig(buf, format="png", dpi=130, bbox_inches="tight",
        facecolor=_BG, edgecolor="none")
234     plt.close(fig)
235     return base64.b64encode(buf.getvalue()).decode()
236
237 def chart_timeline(hourly: dict, anomaly_hours: set):
238     if not hourly: return None
239     hours = list(range(24)); counts = [hourly.get(h, 0) for h in
        hours]
240     colors = ["#e74c3c" if h in anomaly_hours else "#3498db" for h in
         hours]
241     fig, ax = plt.subplots(figsize=(11, 3.5))
242     ax.bar(hours, counts, color=colors, alpha=0.85, width=0.75,
        edgecolor=_DARKER)
243     ax.set_title("       Timeline Alertes 24h", fontsize=11)
244     ax.set_xticks(range(0, 24, 2)); ax.grid(axis="y", alpha=0.2)
245     fig.tight_layout(); return _b64(fig)
246
247 def chart_severity_pie(sev: dict):
248     pairs = [(lbl, sev.get(lbl, 0), col) for lbl, _, _, col, *_ in
        SEV_MATRIX if sev.get(lbl, 0) > 0]
```

```
    if not pairs: return None
    labels = [p[0] for p in pairs]; vals = [p[1] for p in pairs];
       colors = [p[2] for p in pairs]
    fig, ax = plt.subplots(figsize=(4.5, 4.2))
    wedges, _, autotexts = ax.pie(vals, colors=colors, autopct="%1.1f
       %%", startangle=90, wedgeprops=dict(width=0.6, edgecolor=
       _DARKER, linewidth=2))
    for at in autotexts: at.set_color("white"); at.set_fontsize(9)
    ax.legend(wedges, [f"{l} ({v})" for l, v in zip(labels, vals)],
       loc="lower center", bbox_to_anchor=(0.5, -0.22), fontsize=8,
       frameon=False, ncol=2)
    fig.tight_layout(); return _b64(fig)

def chart_top_rules(top_rules: list):
    if not top_rules: return None
    items = top_rules[:8]; labels = [r[:45] + "   " if len(r) > 45
       else r for r, _ in items]
    vals = [v for _, v in items]; colors = plt.cm.Reds(np.linspace
       (0.45, 0.9, len(items)))
    fig, ax = plt.subplots(figsize=(9, max(3.5, len(items) * 0.55)))
    bars = ax.barh(range(len(labels)), vals, color=colors[::-1],
       edgecolor=_DARKER, height=0.65)
    ax.set_yticks(range(len(labels))); ax.set_yticklabels(labels,
       fontsize=8); ax.invert_yaxis()
    ax.set_title("       Top 8 R gles D clench es", fontsize=11);
       ax.grid(axis="x", alpha=0.2)
    fig.tight_layout(); return _b64(fig)

def chart_top_ips(top_ips: list):
    if not top_ips: return None
    labels = [ip for ip, _ in top_ips[:10]]; vals = [v for _, v in
       top_ips[:10]]
    colors = plt.cm.Reds(np.linspace(0.4, 0.9, len(labels)))
    fig, ax = plt.subplots(figsize=(6.5, max(3, len(labels) * 0.5)))
    bars = ax.barh(labels, vals, color=colors, edgecolor=_DARKER,
       height=0.65)
    ax.invert_yaxis(); ax.set_title("       Top IPs Sources",
       fontsize=11); ax.grid(axis="x", alpha=0.2)
    fig.tight_layout(); return _b64(fig)

def chart_mitre(mitre_t: list):
```

```
    if not mitre_t: return None
    tactics = [t for t, _ in mitre_t[:8]]; vals = [v for _, v in
        mitre_t[:8]]
    palette = ["#9b59b6","#8e44ad","#7d3c98","#6c3483","#5b2c6f","#4
        a235a","#6c3483","#7d3c98"][:len(tactics)]
    fig, ax = plt.subplots(figsize=(7, 3.5))
    bars = ax.bar(range(len(tactics)), vals, color=palette, alpha
        =0.85, edgecolor=_DARKER)
    ax.set_xticks(range(len(tactics))); ax.set_xticklabels(tactics,
        rotation=30, ha="right", fontsize=8)
    ax.set_title("       Tactiques MITRE ATT&CK", fontsize=11); ax.
        grid(axis="y", alpha=0.2)
    fig.tight_layout(); return _b64(fig)

def _img(b64, alt=""):
    if not b64: return '<p class="no-data">Donn es insuffisantes</p>
        '
    return f'<img src="{b64}" alt="{alt}">'

#


# 6. SYST ME LLM     R GLE ABSOLUE : PAS DE SUGGESTION BLOCAGE IP
#


_NO_IP_BLOCK = """
       R GLE ABSOLUE ET INTRANSIGEANTE :
Le SOC dispose  D  J   d'un syst me automatique de blocage d'IP via
    Azure Security Group et Shuffle SOAR.
TU NE DOIS SOUS AUCUN PR TEXTE  sugg rer de "bloquer l'IP", "bannir
    l'IP", "blacklister", "bloquer la source", "utiliser iptables", ou
     modifier le pare-feu  r seau.
Toute suggestion  l i  e  au blocage IP est strictement interdite.
Concentre-toi UNIQUEMENT sur : le durcissement applicatif d'Odoo, la
    configuration fine de Wazuh, la politique de mot de passe, l'
    activation du 2FA, ou la gestion des droits.
"""

def _groq(prompt: str, max_tokens: int = 1200) -> str:
    client = Groq(api_key=GROQ_API_KEY)
```

```
    resp   = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[
            {"role": "system", "content": (
                "Tu es un analyste SOC senior sp cialis  en
                   s curit  ERP Odoo et Wazuh IDS.\n"
                "R ponds TOUJOURS en fran ais, format markdown
                   structur .\n"
                "Sois pr cis, concis, orient  action.\n"
                + _NO_IP_BLOCK
            )},
            {"role": "user", "content": prompt},
        ],
        temperature=0.2, max_tokens=max_tokens,
    )
    return resp.choices[0].message.content.strip()

def prompt_executive(stats: dict, anomalies: list) -> str:
    sev   = stats["severity"]
    anom  = "\n".join(f"- [{a['sev']}] {a['type']}: {a['detail']}"
       for a in anomalies) or "Aucune"
    rules = "\n".join(f"- {r} : {c} alertes" for r, c in stats["
       top_rules"][:5])
    mitre = ", ".join(t for t, _ in stats["mitre_tactic"][:5]) or "
       Non identifi "

    return f"""Tu es CISO. G n re un r sum  ex cutif SOC
       professionnel.
{_NO_IP_BLOCK}

## Donn es 24h
| S v rit  | Nb |
|---|---|
|        CRITIQUE ( 12 ) | {sev.get('CRITIQUE',0)} |
|          LEV   (9-11)   | {sev.get('  LEV  ',0)} |
|        MOYEN (6-8)    | {sev.get('MOYEN',0)} |
|        FAIBLE (1-5)   | {sev.get('FAIBLE',0)} |
| Total | {stats['total']} |

## Anomalies
{anom}
```

```
## Top Menaces
{rules}
## Tactiques MITRE
{mitre}

###         tat  Global de S curit
[2-3 phrases]

###        Niveau de Risque : [CRITIQUE /   LEV   / MOD R   / FAIBLE
   ]
[Justification courte]

###     Top 3 Actions Imm diates (48h)
(Rappel strict: AUCUN BLOCAGE IP NE DOIT ETRE SUGG R  ICI)
1. **[Action applicative Odoo ou config Wazuh]**
2. **[Action applicative Odoo ou config Wazuh]**
3. **[Action applicative Odoo ou config Wazuh]**
"""

def prompt_rule_analysis(rule_desc: str, group: list, max_lvl: int,
   sev_lbl: str, sev_col: str) -> str:
    mitre_ids, mitre_tactics = set(), set()
    for a in group:
        mf = a.get("rule", {}).get("mitre", {})
        if isinstance(mf, dict):
            for i in mf.get("id", []): mitre_ids.add(i)
            for t in mf.get("tactic", []): mitre_tactics.add(t)
        elif isinstance(mf, list):
            for item in mf:
                if isinstance(item, str): mitre_ids.add(item)
                elif isinstance(item, dict):
                    if item.get("id"): mitre_ids.add(item["id"])
                    for t in item.get("tactic", []): mitre_tactics.
                       add(t)

    src_ips = list({a.get("data", {}).get("srcip", "") for a in group
        if a.get("data", {}).get("srcip", "") not in ("", "N/A", "
       0.0.0.0")})[:5]

    return f"""Analyste SOC expert Odoo. Analyse cette alerte Wazuh.
{_NO_IP_BLOCK}
```

```
374
375 ## Contexte
376 - **R gle :** {rule_desc}
377 - **Volume :** {len(group)} alertes
378 - **S v rit  max :** Niveau {max_lvl}/15     **{sev_lbl}**
379 - **IPs sources :** {', '.join(src_ips) if src_ips else 'Non
       disponibles'}
380
381 ##          Classification
382 | Champ | Valeur |
383 |---|---|
384 | Type d'attaque | [cat gorie] |
385 | Criticit  | **{sev_lbl}** (Niv {max_lvl}) |
386 | MITRE | {', '.join(sorted(mitre_ids)) or '[   identifier]'} |
387 | Score de risque | [X/10] |
388
389 ##        Analyse
390 [Ce qui s'est pass ]
391
392 ##        Impact Odoo
393 [Impact m tier]
394
395 ##        Rem diation Imm diate (AUCUN BLOCAGE IP)
396 1. **[Config Odoo/Wazuh]** : [D tail]
397 2. **[Config Odoo/Wazuh]** : [D tail]
398
399 ##          Pr vention Long Terme (AUCUN BLOCAGE IP)
400 -        **Wazuh** : [R gle]
401 -        **Odoo** : [Param tre]
402 """
403
404 def template_faible(rule_desc: str, group: list, max_lvl: int) -> str
       :
405     count = len(group)
406     return f"""##          Classification
407 | Champ | Valeur |
408 |---|---|
409 | Type |   vnement   syst me |
410 | Criticit  | **FAIBLE** (Niv {max_lvl}) |
411 | Volume 24h | {count} alertes |
412
```

```
##        Analyse
R gle de faible criticit  d clench e {count} fois. Volume habituel. Aucune action imm diate requise.
"""

#


# 7. EXPORT PDF
#


def html_to_pdf(html_path: str, pdf_path: str) -> bool:
    try:
        from weasyprint import HTML as WP
        WP(filename=html_path).write_pdf(pdf_path)
        size_kb = os.path.getsize(pdf_path) // 1024
        print(f"       PDF g n r   : {pdf_path}  ({size_kb} KB)")
        return True
    except Exception as e:
        print(f"       PDF erreur : {e}")
        return False

#


# 8. ENVOI EMAIL
#


def send_email(html_path: str, pdf_path: str | None, stats: dict, anomalies: list):
    if not (SMTP_USER and SMTP_PASS and EMAIL_TO):
        print("       Email non configur .")
        return

    subj  = f'[SOC] Rapport S curit  {NOW.strftime("%d/%m/%Y")} {stats["total"]} alertes'
    msg = MIMEMultipart("mixed")
    msg["Subject"] = subj; msg["From"] = EMAIL_FROM or SMTP_USER; msg["To"] = EMAIL_TO
```

```
442
443     body = MIMEMultipart("alternative")
444     body.attach(MIMEText("Veuillez trouver ci-joint le rapport PDF
            g  n  r  .", "plain", "utf-8"))
445     with open(html_path, encoding="utf-8") as f:
446         body.attach(MIMEText(f.read(), "html", "utf-8"))
447     msg.attach(body)
448
449     if pdf_path and os.path.exists(pdf_path):
450         with open(pdf_path, "rb") as f: part = MIMEBase("application"
                , "pdf"); part.set_payload(f.read())
451         encoders.encode_base64(part)
452         part.add_header("Content-Disposition", f'attachment; filename
                ="soc_report_{NOW.strftime("%Y%m%d_%H%M")}.pdf"')
453         msg.attach(part)
454
455     try:
456         with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=30) as server
                :
457             server.starttls(); server.login(SMTP_USER, SMTP_PASS)
458             server.sendmail(SMTP_USER, EMAIL_TO, msg.as_string())
459         print(f"       Email envoy     {EMAIL_TO}")
460     except Exception as e:
461         print(f"       Erreur envoi : {e}")
462
463 #
464 # 9. G N RATION HTML / CSS
465 #
466 # ... (CSS complet comme dans votre script)
467 #
468 # 10. MAIN
469 #
470 def main():
```

```
    print("           SOC Report Generator v3.1 (Page de garde + No IP Block)")
    alerts = get_alerts(size=2000)
    stats  = compute_stats(alerts)
    sorted_groups = group_and_sort(alerts)
    anomalies = detect_anomalies(alerts, stats)

    anomaly_hours = {a.get("hour") for a in anomalies if "hour" in a}
    charts = {
        "timeline":  chart_timeline(stats["hourly"], anomaly_hours),
        "severity":  chart_severity_pie(stats["severity"]),
        "top_rules": chart_top_rules(stats["top_rules"]),
        "top_ips":   chart_top_ips(stats["top_ips"]),
        "mitre":     chart_mitre(stats["mitre_tactic"]),
    }

    exec_summary = ""; rule_analyses: dict = {}

    if alerts and GROQ_API_KEY:
        try:
            exec_summary = _groq(prompt_executive(stats, anomalies), max_tokens=900)
            print("       R sum  ex cutif g  n  r  ")
        except Exception as e: print(f"       Erreur Exec : {e}")

        to_analyze = select_for_llm(sorted_groups)
        for i, (rd, grp, mx, sev_lbl, col, emoji, css, prio) in enumerate(to_analyze, 1):
            try:
                rule_analyses[rd] = _groq(prompt_rule_analysis(rd, grp, mx, sev_lbl, col))
                print(f"       R gle IA [{i}/{len(to_analyze)}] : {rd[:40]}...")
            except Exception: pass

    html = generate_html(stats, charts, anomalies, exec_summary, sorted_groups, rule_analyses)
    with open(HTML_FILE, "w", encoding="utf-8") as f: f.write(html)

    print("\  n     Conversion PDF...")
    pdf_ok = html_to_pdf(HTML_FILE, PDF_FILE)
```

```
506
507     print("\  n      Envoi Gmail...")
508     send_email(HTML_FILE, PDF_FILE if pdf_ok else None, stats,
            anomalies)
509     print("        TERMIN ")
510
511 if __name__ == "__main__":
512     main()
```

**Listing 3:** LLM-Powered SOC Report Generator - Main Script

## Dependencies

The script requires the following Python packages:

```
1 pip install requests groq python-dotenv matplotlib numpy markdown2
     weasyprint
```

**Listing 4:** Required Python packages for the SOC Report Generator

## Environment Variables

The script uses the following environment variables (configured via ‘.env‘ file or system environment):

**Table 2:** Environment variables for the SOC Report Generator

| Variable | Default Value | Description |
|---|---|---|
| ES_URL | https://localhost:9200 | OpenSearch endpoint URL |
| ES_USER | admin | OpenSearch username |
| ES_PASS | SecretPassword | OpenSearch password |
| GROQ_API_KEY | - | Groq API key (required) |
| GROQ_MODEL | llama-3.3-70b-versatile | LLM model to use |
| SMTP_HOST | smtp.gmail.com | SMTP server for email delivery |
| SMTP_PORT | 587 | SMTP port |
| SMTP_USER | - | SMTP username (Gmail address) |
| SMTP_PASS | - | SMTP password (Gmail app password) |
| EMAIL_TO | soc-analyst@company.com | Recipient email address |
| ORG_NAME | SOC Wazuh – ERP Odoo | Organization name for report header |

*Source: Personal elaboration based on the SOC reporting system.*

## Scheduled Execution

The script is configured to run automatically every 24 hours via a cron job:

```
# Run SOC report daily at 08:00 AM UTC
0 8 * * * cd ~/soc-llm && python3 soc_report_generator.py >> logs/
   report_$(date +\%Y\%m\%d).log 2>&1
```

**Listing 5:** Cron job for daily SOC report generation

## Report Output

The system generates two output files:

- **HTML Report:** ‘ /soc-llm/reports/socreport.html‘
- **PDF Report:** ‘ /soc-llm/reports/socreport.pdf‘

The PDF is automatically attached to the email and sent to the configured recipient.